Index of Libraries Used

Sl no.	Library	Category
1	***linklist.h***	Singly linked list
2	***stack.h***	LIFO stack
3	***double_ll.h***	Doubly linked list
4	***queue.h***	FIFO queue
5	***list.h***	Dynamic array
6	***bst.h***	Binary Search Tree (AVL-balanced)
7	***heap.h***	Binary max-heap
8	***hash.h***	Chained hash table
9	***set.h***	Hash set
10	***sort.h***	Sorting algorithms

Libraries Used — Description

1. ***linklist.h*** — Singly linked list. Generic, ***void****-based singly linked list providing head/indexed insert and delete operations, an optional per-element free_ele destructor, and ***O(n)*** traversal helpers. Forms the underlying storage for ***stack.h.***
2. ***stack.h*** — LIFO stack. A thin LIFO wrapper (init, push, pop, peek, isempty, free) around ***linklist.h***: ***stack_t*** is simply a ***linklist_t***, and every push/pop happens at the head of the list.
3. ***double_ll.h*** — Doubly linked list. Generic doubly linked list with ***O(1)*** head and tail insert/delete and bidirectional node links (***dll_node***), used as the underlying storage for ***queue.h***.
4. ***queue.h*** — FIFO queue. A thin FIFO wrapper (init, enqueue, dequeue, peek, isempty, size, free) around ***double_ll.h***: ***queue_t*** is a ***double_ll_t***, with enqueue inserting at the tail and dequeue removing from the head.
5. ***list.h*** — Dynamic array. A generic, growable dynamic array (init, add, set, get, size, free) that stores elements as raw byte copies and doubles its capacity on growth.
6. ***bst.h*** — Binary Search Tree (AVL-balanced). A self-balancing BST — insert/delete perform AVL rotations to keep the tree height-balanced — supporting insert, get, contains and delete, all ***O(log n)***, driven by a user-supplied comparator.
7. ***heap.h*** — Binary max-heap. An array-free, pointer-based binary max-heap (init, insert, peek, popmax). Ordering is fully customizable via a comparator, letting it double as a min-heap or a priority queue.
8. ***hash.h*** — Chained hash table. A separate-chaining hash table (init, put, get, remove) built on an array of ***linklist.h*** buckets, with a built-in ***djb2***-style byte hash and pluggable custom hash/comparator functions for non-trivial key types such as C strings. Internally underlies ***set.h***.
9. ***set.h*** — Hash set. A minimal hash-set wrapper (init, insert, contains, remove, free) built directly on top of ***hash.h*** (a set is just a hash table whose values are an unused placeholder byte).
10. ***sort.h*** — Sorting algorithms. A collection of independent, qsort-style generic (***void*, element-size, comparator***) sorting algorithms — insertion, selection, merge, bubble and quick sort — each enabled by its own macro.

Library — linklist.h (Singly linked list)

```
// linklist.h

#ifndef LINKLIST_H
#define LINKLIST_H

#include <stddef.h> //size_t

typedef struct linklist_t linklist_t;

typedef struct ll_node ll_node;

int ll_init(linklist_t *l, size_t elem_sz, void (*free_ele)(void *));
void ll_free(linklist_t *l);

int ll_isempty(const linklist_t *l);
size_t ll_size(const linklist_t *l);

int ll_insert(linklist_t *l, size_t i, const void *elem);
int ll_inserthead(linklist_t *l, const void *elem);
int ll_inserttail(linklist_t *l, const void *elem);

int ll_delete(linklist_t *l, size_t i, void *out_elem);
int ll_deletehead(linklist_t *l, void *out_elem);
int ll_deletetail(linklist_t *l, void *out_elem);

int ll_getelem(const linklist_t *l, size_t i, void *out_elem);
int ll_gethead(const linklist_t *l, void *out_elem);
int ll_gettail(const linklist_t *l, void *out_elem);

#ifdef LL_IMPLEMENTATION

struct ll_node {
    void *data;
    struct ll_node *next;
};

struct linklist_t {
    ll_node *head;
    size_t elem_sz;
    void (*__free_ele)(void *);
};
#include <stdlib.h>
#include <string.h>
static void __ll_free_node(ll_node *n, void (*free_ele)(void *)) {
    if (!n)
        return;
    if (n->data) {
        if (free_ele)
            free_ele(n->data);
        free(n->data);
    }
    free(n);
}
static ll_node *__ll_create_node(const void *elem, size_t elem_sz) {
    ll_node *n = (ll_node *)malloc(sizeof(ll_node));
    if (!n)
        return NULL;
    n->data = malloc(elem_sz);
    if (!n->data) {
        __ll_free_node(n, NULL);
        return NULL;
    }
    memcpy(n->data, elem, elem_sz);
    n->next = NULL;
    return n;
}
int ll_init(linklist_t *l, size_t elem_sz, void (*free_ele)(void *)) {
    if (!l || elem_sz == 0)
        return 0;

    l->head = NULL;
    l->elem_sz = elem_sz;
    l->__free_ele = free_ele;
    return 1;
}
static void __ll_node_rec_delete(ll_node *node, void (*free_ele)(void *)) {
    if (!node)
```

```
        return;
    __ll_node_rec_delete(node->next, free_ele);
    __ll_free_node(node, free_ele);
}

void ll_free(linklist_t *l) {
    if (!l)
        return;

    __ll_node_rec_delete(l->head, l->__free_ele);

    l->head = NULL;
    return;
}

int ll_isempty(const linklist_t *l) { return !l || !l->head; }

size_t ll_size(const linklist_t *l) {
    size_t s = 0;
    for (const ll_node *cur = l->head; cur; cur = cur->next)
        s++;
    return s;
}

int ll_inserthead(linklist_t *l, const void *elem) {
    if (!l)
        return 0;

    ll_node *n = __ll_create_node(elem, l->elem_sz);
    if (!n)
        return 0;

    n->next = l->head;
    l->head = n;
    return 1;
}
int ll_inserttail(linklist_t *l, const void *elem) {
    if (!l)
        return 0;

    if (!l->head)
        return ll_inserthead(l, elem);

    ll_node *n = __ll_create_node(elem, l->elem_sz);
    if (!n)
        return 0;

    ll_node *prev;
    for (prev = l->head; prev->next; prev = prev->next)
        ;

    prev->next = n;
    return 1;
}
int ll_insert(linklist_t *l, size_t i, const void *elem) {
    if (!l)
        return 0;

    if (i == 0)
        return ll_inserthead(l, elem);

    ll_node *prev;
    for (prev = l->head; prev && --i; prev = prev ? prev->next : NULL)
        ;
    if (!prev)
        return 0;

    ll_node *n = __ll_create_node(elem, l->elem_sz);
    if (!n)
        return 0;

    n->next = prev->next;
    prev->next = n;
    return 1;
}
int ll_deletehead(linklist_t *l, void *out_elem) {
    if (ll_isempty(l) || !out_elem)
        return 0;

    ll_node *tmp = l->head;
```

```
    l->head = tmp->next;
    memcpy(out_elem, tmp->data, l->elem_sz);
    __ll_free_node(tmp, l->__free_ele);
    return 1;
}
int ll_deletetail(linklist_t *l, void *out_elem) {
    if (ll_isempty(l) || !out_elem)
        return 0;

    if (l->head->next == NULL)
        return ll_deletehead(l, out_elem);

    ll_node *prev, *tmp;
    for (prev = l->head; prev->next->next; prev = prev->next)
        ;

    tmp = prev->next;
    prev->next = NULL;
    memcpy(out_elem, tmp->data, l->elem_sz);
    __ll_free_node(tmp, l->__free_ele);
    return 1;
}
int ll_delete(linklist_t *l, size_t i, void *out_elem) {
    if (ll_isempty(l) || !out_elem)
        return 0;

    if (i == 0)
        return ll_deletehead(l, out_elem);

    ll_node *prev, *tmp;
    for (prev = l->head; prev && --i; prev = prev ? prev->next : NULL)
        ;
    if (!prev)
        return 0;

    tmp = prev->next;
    if (!tmp)
        return 0;
    prev->next = tmp->next;
    memcpy(out_elem, tmp->data, l->elem_sz);
    __ll_free_node(tmp, l->__free_ele);
    return 1;
}

int ll_gethead(const linklist_t *l, void *out_elem) {
    if (ll_isempty(l) || !out_elem)
        return 0;
    memcpy(out_elem, l->head->data, l->elem_sz);
    return 1;
}

int ll_gettail(const linklist_t *l, void *out_elem) {
    if (ll_isempty(l) || !out_elem)
        return 0;
    const ll_node *last;
    for (last = l->head; last->next; last = last->next)
        ;
    memcpy(out_elem, last->data, l->elem_sz);
    return 1;
}

int ll_getelem(const linklist_t *l, size_t i, void *out_elem) {
    if (ll_isempty(l) || !out_elem)
        return 0;

    if (i == 0)
        return ll_gethead(l, out_elem);

    const ll_node *n;
    for (n = l->head; n && i--; n = n ? n->next : NULL)
        ;
    if (!n)
        return 0;
    memcpy(out_elem, n->data, l->elem_sz);
    return 1;
}

#endif // LL_IMPLEMENTATION

#endif // LINKLIST_H
```

Library — stack.h (LIFO stack)

```
// stack.h
#ifndef STACK_H
#define STACK_H

#include <stddef.h> // size_t

typedef struct linklist_t stack_t;

int stack_init(stack_t *s, size_t elem_sz, void (*free_ele)(void *));
void stack_free(stack_t *s);

int stack_isempty(const stack_t *s);

int stack_push(stack_t *s, const void *elem);
int stack_pop(stack_t *s, void *out);
int stack_peek(const stack_t *s, void *out);

#ifdef STACK_IMPLEMENTATION

#ifdef STACK_IMPLEMENTATION
#ifndef LL_IMPLEMENTATION
#define LL_IMPLEMENTATION
#endif // LL_IMPLEMENTATION
#endif // STACK_IMPLEMENTATION

#include "linklist.h"

int stack_init(stack_t *s, size_t elem_sz, void (*free_ele)(void *)) {
    return ll_init(s, elem_sz, free_ele);
}

void stack_free(stack_t *s) { ll_free(s); }

int stack_isempty(const stack_t *s) { return ll_isempty(s); }

int stack_push(stack_t *s, const void *elem) { return ll_inserthead(s, elem); }

int stack_pop(stack_t *s, void *out) { return ll_deletehead(s, out); }

int stack_peek(const stack_t *s, void *out) { return ll_gethead(s, out); }

#endif // STACK_IMPLEMENTATION

#endif // STACK_H
```

Library — double_ll.h (Doubly linked list)

```
// double_ll.h

#ifndef DOUBLE_LL_H
#define DOUBLE_LL_H

#include <stddef.h> // size_t

typedef struct dll_node dll_node;
typedef struct double_ll_t double_ll_t;

int dll_init(double_ll_t *l, size_t elem_sz, void (*free_ele)(void *));
void dll_free(double_ll_t *l);

int dll_isempty(const double_ll_t *l);
size_t dll_size(const double_ll_t *l);

dll_node *dll_insert(double_ll_t *l, size_t i, const void *elem);
dll_node *dll_inserthead(double_ll_t *l, const void *elem);
dll_node *dll_inserttail(double_ll_t *l, const void *elem);

int dll_delete(double_ll_t *l, size_t i, void *out_elem);
int dll_deletehead(double_ll_t *l, void *out_elem);
int dll_deletetail(double_ll_t *l, void *out_elem);
int dll_delete_node(double_ll_t *l, dll_node *n, void *out_elem);

int dll_getelem(const double_ll_t *l, size_t i, void *out_elem);
int dll_gethead(const double_ll_t *l, void *out_elem);
int dll_gettail(const double_ll_t *l, void *out_elem);

#ifdef DLL_IMPLEMENTATION

struct dll_node {
    void *data;
    struct dll_node *next;
    struct dll_node *prev;
};

struct double_ll_t {
    dll_node *head;
    dll_node *tail;
    size_t elem_sz;
    void (*__free_ele)(void *);
};

#include <stdlib.h>
#include <string.h>

static void __dll_free_node(dll_node *n, void (*free_ele)(void *)) {
    if (!n)
        return;
    if (n->data) {
        if (free_ele)
            free_ele(n->data);
        free(n->data);
    }
    free(n);
}

static dll_node *__dll_create_node(const void *elem, size_t elem_sz) {
    dll_node *n = (dll_node *)malloc(sizeof(dll_node));
    if (!n)
        return NULL;
    n->data = malloc(elem_sz);
    if (!n->data) {
        __dll_free_node(n, NULL);
        return NULL;
    }
    memcpy(n->data, elem, elem_sz);
    n->next = NULL;
    n->prev = NULL;
    return n;
}

int dll_init(double_ll_t *l, size_t elem_sz, void (*free_ele)(void *)) {
    if (!l || elem_sz == 0)
        return 0;
```

```
    l->head = NULL;
    l->tail = NULL;
    l->elem_sz = elem_sz;
    l->__free_ele = free_ele;
    return 1;
}

static void __dll_node_rec_delete(dll_node *node, void (*free_ele)(void *)) {
    if (!node)
        return;
    __dll_node_rec_delete(node->next, free_ele);
    __dll_free_node(node, free_ele);
}

void dll_free(double_ll_t *l) {
    if (!l)
        return;

    __dll_node_rec_delete(l->head, l->__free_ele);
    l->head = NULL;
    l->tail = NULL;
    l->elem_sz = 0;
    l->__free_ele = NULL;
}

int dll_isempty(const double_ll_t *l) { return !l || !l->head; }

size_t dll_size(const double_ll_t *l) {
    size_t s = 0;
    for (const dll_node *cur = l->head; cur; cur = cur->next)
        s++;
    return s;
}

dll_node *dll_inserthead(double_ll_t *l, const void *elem) {
    if (!l)
        return NULL;

    dll_node *n = __dll_create_node(elem, l->elem_sz);
    if (!n)
        return NULL;

    n->next = l->head;
    n->prev = NULL;
    if (l->head) {
        l->head->prev = n;
    } else {
        l->tail = n;
    }
    l->head = n;
    return n;
}

dll_node *dll_inserttail(double_ll_t *l, const void *elem) {
    if (!l)
        return NULL;
    if (!l->head)
        return dll_inserthead(l, elem);

    dll_node *n = __dll_create_node(elem, l->elem_sz);
    if (!n)
        return NULL;

    l->tail->next = n;
    n->prev = l->tail;
    l->tail = n;
    return n;
}

dll_node *dll_insert(double_ll_t *l, size_t i, const void *elem) {
    if (!l)
        return NULL;
    if (i == 0)
        return dll_inserthead(l, elem);

    dll_node *cur = l->head;
    while (cur && i > 1) {
        cur = cur->next;
        i--;
    }
```

```
    if (!cur)
        return NULL;

    dll_node *n = __dll_create_node(elem, l->elem_sz);
    if (!n)
        return NULL;

    n->next = cur->next;
    n->prev = cur;
    if (cur->next) {
        cur->next->prev = n;
    } else {
        l->tail = n;
    }
    cur->next = n;
    return n;
}

int dll_delete_node(double_ll_t *l, dll_node *n, void *out_elem) {
    if (!l || !n)
        return 0;
    if (n == l->head) {
        l->head = n->next;
    }
    if (n == l->tail) {
        l->tail = n->prev;
    }
    if (n->prev) {
        n->prev->next = n->next;
    }
    if (n->next) {
        n->next->prev = n->prev;
    }
    if (out_elem) {
        memcpy(out_elem, n->data, l->elem_sz);
    }
    __dll_free_node(n, l->__free_ele);
    return 1;
}

int dll_deletehead(double_ll_t *l, void *out_elem) {
    if (dll_isempty(l) || !out_elem)
        return 0;
    memcpy(out_elem, l->head->data, l->elem_sz);
    return dll_delete_node(l, l->head, out_elem);
}

int dll_deletetail(double_ll_t *l, void *out_elem) {
    if (dll_isempty(l) || !out_elem)
        return 0;
    memcpy(out_elem, l->tail->data, l->elem_sz);
    return dll_delete_node(l, l->tail, out_elem);
}

int dll_delete(double_ll_t *l, size_t i, void *out_elem) {
    if (dll_isempty(l) || !out_elem)
        return 0;
    if (i == 0)
        return dll_deletehead(l, out_elem);

    dll_node *cur = l->head;
    while (cur && i > 0) {
        cur = cur->next;
        i--;
    }
    if (!cur)
        return 0;
    memcpy(out_elem, cur->data, l->elem_sz);
    return dll_delete_node(l, cur, out_elem);
}

int dll_gethead(const double_ll_t *l, void *out_elem) {
    if (dll_isempty(l) || !out_elem)
        return 0;
    memcpy(out_elem, l->head->data, l->elem_sz);
    return 1;
}

int dll_gettail(const double_ll_t *l, void *out_elem) {
    if (dll_isempty(l) || !out_elem)
```

```
        return 0;
    memcpy(out_elem, l->tail->data, l->elem_sz);
    return 1;
}

int dll_getelem(const double_ll_t *l, size_t i, void *out_elem) {
    if (dll_isempty(l) || !out_elem)
        return 0;
    if (i == 0)
        return dll_gethead(l, out_elem);
    dll_node *cur = l->head;
    while (cur && i > 0) {
        cur = cur->next;
        i--;
    }
    if (!cur)
        return 0;

    memcpy(out_elem, cur->data, l->elem_sz);
    return 1;
}

#endif // DLL_IMPLEMENTATION
#endif // DOUBLE_LL_H
```

Library — queue.h (FIFO queue)

```
// queue.h

#ifndef QUEUE_H
#define QUEUE_H

#include <stddef.h> // size_t

typedef struct double_ll_t queue_t;

int q_init(queue_t *q, size_t elem_sz, void (*free_ele)(void *));
void q_free(queue_t *q);
int q_isempty(const queue_t *q);
size_t q_size(const queue_t *q);
int enqueue(queue_t *q, const void *elem);
int dequeue(queue_t *q, void *out);
int peek(const queue_t *q, void *out);

#ifdef QUEUE_IMPLEMENTATION

#ifdef QUEUE_IMPLEMENTATION
#ifndef DLL_IMPLEMENTATION
#define DLL_IMPLEMENTATION
#endif // DLL_IMPLEMENTATION
#endif // QUEUE_IMPLEMENTATION

#include "double_ll.h"

int q_init(queue_t *q, size_t elem_sz, void (*free_ele)(void *)) {
    return dll_init(q, elem_sz, free_ele);
}

void q_free(queue_t *q) { dll_free(q); }

int q_isempty(const queue_t *q) { return dll_isempty(q); }

size_t q_size(const queue_t *q) { return dll_size(q); }

int enqueue(queue_t *q, const void *elem) {
    return dll_inserttail(q, elem) != NULL;
}

int dequeue(queue_t *q, void *out) { return dll_deletehead(q, out); }

int peek(const queue_t *q, void *out) { return dll_gethead(q, out); }
#endif // QUEUE_IMPLEMENTATION
#endif // QUEUE_H
```

Library — list.h (Dynamic array)

```
// list.h

#ifndef LIST_H
#define LIST_H

#include <stddef.h> // size_t

typedef struct list_t list_t;

int list_init(list_t *l, size_t initial_cap, size_t elem_sz,
              void (*free_ele)(void *));
void list_free(list_t *l);

size_t list_size(const list_t *l);

int add(list_t *l, const void *elem);

int set(list_t *l, size_t i, const void *elem);

int get(const list_t *l, size_t i, void *out);

#ifdef LIST_IMPLEMENTATION

struct list_t {
    unsigned char *data;
    size_t cap;
    size_t size;
    size_t elem_sz;
    void (*__free_ele)(void *);
};

#include <stdlib.h>
#include <string.h>

int list_init(list_t *l, size_t initial_cap, size_t elem_sz,
              void (*free_ele)(void *)) {
    if (!l || initial_cap == 0 || elem_sz == 0)
        return 0;

    l->data = calloc(initial_cap, elem_sz);
    if (!l->data)
        return 0;

    l->cap = initial_cap;
    l->size = 0;
    l->elem_sz = elem_sz;
    l->__free_ele = free_ele;
    return 1;
}

void list_free(list_t *l) {
    if (!l)
        return;

    if (l->data) {
        if (l->__free_ele)
            for (size_t i = 0; i < l->size; i++)
                l->__free_ele(&l->data[i * l->elem_sz]);
        free(l->data);
    }

    l->data = NULL;
    l->cap = 0;
    l->size = 0;
    l->elem_sz = 0;
    l->__free_ele = NULL;
}

size_t list_size(const list_t *l) {
    if (!l)
        return 0;
    return l->size;
}

static int __grow_list(list_t *l) {
    size_t new_cap = 2 * l->cap;
```

```
    void *new_data = reallocarray(l->data, new_cap, l->elem_sz);
    if (!new_data)
        return 0;

    l->data = new_data;
    l->cap = new_cap;
    return 1;
}

int add(list_t *l, const void *elem) {
    if (!l || !elem)
        return 0;
    if (l->size == l->cap && !__grow_list(l))
        return 0;

    memcpy(&l->data[l->size++ * l->elem_sz], elem, l->elem_sz);
    return 1;
}

int set(list_t *l, size_t i, const void *elem) {
    if (!l || !elem)
        return 0;
    if (i > l->size)
        return 0;
    if (i == l->size)
        return add(l, elem);

    if (l->__free_ele)
        l->__free_ele(&l->data[i * l->elem_sz]);
    memcpy(&l->data[i * l->elem_sz], elem, l->elem_sz);
    return 1;
}

int get(const list_t *l, size_t i, void *out) {
    if (!l || !out)
        return 0;
    if (i >= l->size)
        return 0;

    memcpy(out, &l->data[i * l->elem_sz], l->elem_sz);
    return 1;
}

#endif // LIST_IMPLEMENTATION
#endif // LIST_H
```

Library — bst.h (Binary Search Tree (AVL-balanced))

```
// bst.h

#ifndef BST_H
#define BST_H

#include <stddef.h>

typedef struct bst_node bst_node;
typedef struct bst_t bst_t;

int bst_init(bst_t *b, size_t elem_sz, int (*cmp)(const void *, const void *),
             void (*free_ele)(void *));
void bst_free(bst_t *b);

int bst_insert(bst_t *b, const void *elem);
int bst_delete(bst_t *b, const void *elem);
int bst_contains(const bst_t *b, const void *elem);
int bst_get(const bst_t *b, const void *elem, void *out);

#ifdef BST_IMPLEMENTATION

struct bst_t {
    bst_node *root;
    size_t elem_sz;
    int (*cmp)(const void *, const void *);
    void (*free_ele)(void *);
};

struct bst_node {
    void *val;
    bst_node *left;
    bst_node *right;
    size_t __ht; // AVL implementation
};

#include <stdlib.h>
#include <string.h>

int bst_init(bst_t *b, size_t elem_sz, int (*cmp)(const void *, const void *),
             void (*free_ele)(void *)) {
    if (elem_sz == 0 || !cmp || !b)
        return 0;
    b->root = NULL;
    b->elem_sz = elem_sz;
    b->cmp = cmp;
    b->free_ele = free_ele;
    return 1;
}

static void __free_bst_node(bst_node *n, void (*free_ele)(void *)) {
    if (!n)
        return;
    if (free_ele)
        free_ele(n->val);
    free(n->val);
    free(n);
}

static void __rec_free(bst_node *n, void (*free_ele)(void *)) {
    if (!n)
        return;
    __rec_free(n->left, free_ele);
    __rec_free(n->right, free_ele);
    __free_bst_node(n, free_ele);
}

void bst_free(bst_t *b) {
    if (!b)
        return;
    __rec_free(b->root, b->free_ele);
    b->root = NULL;
    b->elem_sz = 0;
    b->cmp = NULL;
    b->free_ele = NULL;
}

static bst_node *__bst_create_node(const void *elem, size_t elem_sz) {
```

```
    bst_node *n = (bst_node *)malloc(sizeof(bst_node));
    if (!n)
        return NULL;
    void *val = malloc(elem_sz);
    if (!val) {
        free(n);
        return NULL;
    }
    n->val = val;
    memcpy(n->val, elem, elem_sz);
    n->left = NULL;
    n->right = NULL;
    n->__ht = 1;
    return n;
}

static size_t __get_ht(const bst_node *n) {
    if (!n)
        return 0;
    return n->__ht;
}

static void __update_ht(bst_node *n) {
    if (!n)
        return;
    size_t a, b;
    a = __get_ht(n->left);
    b = __get_ht(n->right);
    n->__ht = (a > b ? a : b) + 1;
}

static int __bf(const bst_node *n) {
    if (!n)
        return 0;
    return __get_ht(n->left) - __get_ht(n->right);
}

static bst_node *__rot_left(bst_node *n) {
    /* left rotation: pivot is right child */
    bst_node *x, *s;

    x = n->right;
    if (!x)
        return n;
    s = x->left;

    x->left = n;
    n->right = s;

    __update_ht(n);
    __update_ht(x);

    return x;
}

static bst_node *__rot_right(bst_node *n) {
    /* right rotation: pivot is left child */
    bst_node *x, *s;
    x = n->left;
    if (!x)
        return n;
    s = x->right;
    x->right = n;
    n->left = s;
    __update_ht(n);
    __update_ht(x);
    return x;
}

static bst_node *__rebal(bst_node *n) {
    __update_ht(n);

    int bf = __bf(n);

    // left heavy
    if (bf > 1) {
        if (__bf(n->left) < 0)
            n->left = __rot_left(n->left); // LR
        return __rot_right(n);             // LL
    }
```

```
    // right heavy
    if (bf < -1) {
        if (__bf(n->right) > 0)
            n->right = __rot_right(n->right); // RL
        return __rot_left(n);                 // RR
    }

    return n;
}

static bst_node *__rec_insert(bst_node *n, const void *elem, size_t elem_sz,
                              int (*cmp)(const void *, const void *)) {
    if (!n)
        return __bst_create_node(elem, elem_sz);

    int k = cmp(elem, n->val);
    if (k == 0)
        return n;
    if (k > 0)
        n->right = __rec_insert(n->right, elem, elem_sz, cmp);
    if (k < 0)
        n->left = __rec_insert(n->left, elem, elem_sz, cmp);
    return __rebal(n);
}

int bst_insert(bst_t *b, const void *elem) {
    if (!b)
        return 0;

    b->root = __rec_insert(b->root, elem, b->elem_sz, b->cmp);
    return 1; // Doesnt guarantee insert success
}

static int __rec_contains(const bst_node *n, const void *elem,
                          int (*cmp)(const void *, const void *)) {
    if (!n)
        return 0;
    int k = cmp(elem, n->val);
    if (k == 0)
        return 1;
    if (k > 0)
        return __rec_contains(n->right, elem, cmp);
    if (k < 0)
        return __rec_contains(n->left, elem, cmp);
    __builtin_unreachable();
}

int bst_contains(const bst_t *b, const void *elem) {
    if (!b)
        return 0;
    return __rec_contains(b->root, elem, b->cmp);
}

static int __rec_get(const bst_node *n, const void *elem,
                          int (*cmp)(const void *, const void *), void *out, size_t elem_sz) {
    if (!n)
        return 0;
    int k = cmp(elem, n->val);
    if (k == 0) {
        memcpy(out, n->val, elem_sz);
        return 1;
    }
    if (k > 0)
        return __rec_get(n->right, elem, cmp, out, elem_sz);
    if (k < 0)
        return __rec_get(n->left, elem, cmp, out, elem_sz);
    __builtin_unreachable();
}

int bst_get(const bst_t *b, const void *elem, void *out) {
    if (!b)
        return 0;
    return __rec_get(b->root, elem, b->cmp, out, b->elem_sz);
}

static bst_node *__rec_delete(bst_node *n, const void *elem, size_t elem_sz,
                              int (*cmp)(const void *, const void *),
                              void (*free_ele)(void *)) {
    if (!n)
```

```
        return NULL;

    int k = cmp(elem, n->val);

    if (k > 0)
        n->right = __rec_delete(n->right, elem, elem_sz, cmp, free_ele);
    else if (k < 0)
        n->left = __rec_delete(n->left, elem, elem_sz, cmp, free_ele);
    else {
        if (!n->right || !n->left) {
            bst_node *c;
            if (!n->left && !n->right)
                c = NULL;
            else
                c = !n->left ? n->right : n->left;
            __free_bst_node(n, free_ele);
            return c;
        }
        bst_node *suc = n->right;
        while (suc->left)
            suc = suc->left;

        if (free_ele)
            free_ele(n->val);
        memcpy(n->val, suc->val, elem_sz);
        n->right = __rec_delete(n->right, suc->val, elem_sz, cmp, free_ele);
    }

    return __rebal(n);
}

int bst_delete(bst_t *b, const void *elem) {
    if (!b)
        return 0;

    b->root = __rec_delete(b->root, elem, b->elem_sz, b->cmp, b->free_ele);
    return 1; // Doesnt guarantee success
}

#endif // BST_IMPLEMENTATION
#endif // BST_H
```

Library — heap.h (Binary max-heap)

```
// heap.h

#ifndef HEAP_H
#define HEAP_H

#include <stddef.h> // size_t

typedef struct heap_node hp_node;
typedef struct heap_t hp_t;

int hp_init(hp_t *h, size_t elem_sz, int (*cmp)(const void *, const void *),
            void (*free_ele)(void *));
void hp_free(hp_t *h);

int hp_insert(hp_t *h, const void *elem);
int hp_peek(const hp_t *h, void *out);
int hp_popmax(hp_t *h, void *out);

#ifdef HEAP_IMPLEMENTATION

struct heap_t {
    hp_node *root;
    size_t elem_sz;
    size_t size;
    int (*cmp)(const void *, const void *);
    void (*free_ele)(void *);
};

struct heap_node {
    void *val;
    hp_node *left;
    hp_node *right;
    hp_node *parent;
};

#include <math.h>
#include <stdlib.h>
#include <string.h>

int hp_init(hp_t *h, size_t elem_sz, int (*cmp)(const void *, const void *),
            void (*free_ele)(void *)) {
    if (!h || !cmp || elem_sz == 0)
        return 0;

    h->root = NULL;
    h->elem_sz = elem_sz;
    h->size = 0;
    h->cmp = cmp;
    h->free_ele = free_ele;
    return 1;
}

static void __hp_free_node(hp_node *n, void (*free_ele)(void *)) {
    if (!n)
        return;
    if (free_ele)
        free_ele(n->val);
    free(n->val);
    free(n);
}

static void __rec_free(hp_node *n, void (*free_ele)(void *)) {
    if (!n)
        return;
    __rec_free(n->left, free_ele);
    __rec_free(n->right, free_ele);
    __hp_free_node(n, free_ele);
}

void hp_free(hp_t *h) {
    if (!h)
        return;
    __rec_free(h->root, h->free_ele);
    h->root = NULL;
    h->elem_sz = 0;
    h->size = 0;
    h->cmp = NULL;
```

```
    h->free_ele = NULL;
}

static hp_node *__hp_create_node(const void *elem, size_t elem_sz) {
    /* allocate node struct correctly */
    hp_node *n = (hp_node *)malloc(sizeof(hp_node));
    if (!n)
        return NULL;
    void *val = malloc(elem_sz);
    if (!val) {
        free(n);
        return NULL;
    }
    n->val = val;
    memcpy(n->val, elem, elem_sz);
    n->left = NULL;
    n->right = NULL;
    n->parent = NULL;
    return n;
}

static void __swap_val(hp_node *a, hp_node *b) {
    void *t = a->val;
    a->val = b->val;
    b->val = t;
}

static void __bubble_up(hp_node *n, int (*cmp)(const void *, const void *)) {
    while (n->parent && cmp(n->val, n->parent->val) > 0) {
        __swap_val(n, n->parent);
        n = n->parent;
    }
}

static void __bubble_down(hp_node *n, int (*cmp)(const void *, const void *)) {
    while (1) {
        hp_node *l = n;

        if (n->left && cmp(n->left->val, l->val) > 0)
            l = n->left;
        if (n->right && cmp(n->right->val, l->val) > 0)
            l = n->right;
        if (l == n)
            break;

        __swap_val(l, n);
        n = l;
    }
}

static hp_node *__get_node(const hp_node *n, size_t i, size_t d) {
    if (d == 0)
        return (hp_node *)n;

    size_t div = (size_t)1 << (d - 1); // 2 ^ (d - 1)

    if (i >= div)
        return __get_node(n->right, i - div, d - 1); //  Right subtree
    else
        return __get_node(n->left, i, d - 1); // Left subtree
}

int hp_insert(hp_t *h, const void *elem) {
    if (!h || !elem)
        return 0;
    hp_node *n = __hp_create_node(elem, h->elem_sz);
    if (!n)
        return 0;

    h->size++;

    if (!h->root) {
        h->root = n;
        return 1;
    }

    size_t k = h->size / 2;
    size_t d = floor(log2(k));
    size_t o = k - ((size_t)1 << d);
```

```
    hp_node *p = __get_node(h->root, o, d);

    n->parent = p;

    if (!p->left)
        p->left = n;
    else
        p->right = n;

    __bubble_up(n, h->cmp);
    return 1;
}

int hp_peek(const hp_t *h, void *out) {
    if (!h || !h->root || !out)
        return 0;
    memcpy(out, h->root->val, h->elem_sz);
    return 1;
}

int hp_popmax(hp_t *h, void *out) {
    if (!h || !h->root || !out)
        return 0;
    memcpy(out, h->root->val, h->elem_sz);

    if (h->size == 1) {
        __hp_free_node(h->root, h->free_ele);
        h->root = NULL;
        h->size = 0;
        return 1;
    }

    size_t k = h->size;
    size_t d = floor(log2(k));
    size_t o = k - ((size_t)1 << d);

    hp_node *last = __get_node(h->root, o, d);
    __swap_val(h->root, last);
    hp_node *p = last->parent;

    if (p->left == last)
        p->left = NULL;
    else
        p->right = NULL;

    __hp_free_node(last, h->free_ele);
    h->size--;

    __bubble_down(h->root, h->cmp);
    return 1;
}

#endif // HEAP_IMPLEMENTATION
#endif // HEAP_H
```

Library — hash.h (Chained hash table)

```
// hash.h

#ifndef HASH_H
#define HASH_H

#include <stddef.h>

typedef struct hash_t hash_t;

int ht_init(hash_t *h, size_t key_sz, size_t val_sz, size_t buckets,
            void (*key_free)(void *), void (*val_free)(void *));
void set_hash(hash_t *h, size_t (*hash)(const void *, size_t));
void set_cmp(hash_t *h, int (*cmp)(const void *, const void *, size_t));
void ht_free(hash_t *h);

int ht_put(hash_t *h, const void *key, const void *val);
int ht_get(const hash_t *h, const void *key, void *out);
int ht_rem(hash_t *h, const void *key);

#ifdef HASH_IMPLEMENTATION

#ifdef HASH_IMPLEMENTATION
#ifndef LL_IMPLEMENTATION
#define LL_IMPLEMENTATION
#endif // LL_IMPLEMENTATION
#endif // HASH_IMPLEMENTATION

#include "linklist.h"

struct __ctx {
    void (*key_free)(void *);
    void (*val_free)(void *);
};

typedef struct __entry_t __entry_t;
struct __entry_t {
    void *key;
    void *val;
    struct __ctx *ctx;
};

struct hash_t {
    linklist_t *arr; // hash_t is an array of linked-list of __entry_t
    size_t key_sz;
    size_t val_sz;
    size_t buckets;
    struct __ctx __ctx;
    size_t (*hash_fn)(const void *, size_t);
    int (*cmp_fn)(const void *, const void *, size_t);
};

#include <stdlib.h>
#include <string.h>

// djb2 hash
size_t __hash(const void *key, size_t size) {
    const unsigned char *p = key;
    size_t h = 5381;

    for (size_t i = 0; i < size; i++)
        h = ((h << 5) + h) + p[i];
    return h;
}

int __create_entry(const void *key, const void *val, size_t key_sz,
                   size_t val_sz, __entry_t *out, struct __ctx *ctx) {
    if (!key || !val || !out)
        return 0;

    out->key = malloc(key_sz);
    if (!out->key)
        return 0;
    memcpy(out->key, key, key_sz);

    out->val = malloc(val_sz);
    if (!out->val) {
        free(out->key);
```

```
        return 0;
    }
    memcpy(out->val, val, val_sz);

    out->ctx = ctx;

    return 1;
}

void __destroy_entry(void *p) {
    __entry_t *entry = (__entry_t *)p;

    if (!entry)
        return;
    if (entry->ctx->key_free)
        entry->ctx->key_free(entry->key);
    free(entry->key);
    if (entry->ctx->val_free)
        entry->ctx->val_free(entry->val);
    free(entry->val);
}

int ht_init(hash_t *h, size_t key_sz, size_t val_sz, size_t buckets,
            void (*key_free)(void *), void (*val_free)(void *)) {
    if (!h || buckets == 0 || key_sz == 0 || val_sz == 0)
        return 0;

    linklist_t *arr = (linklist_t *)calloc(buckets, sizeof(linklist_t));
    if (!arr)
        return 0;

    for (size_t i = 0; i < buckets; ++i)
        if (!ll_init(&arr[i], sizeof(__entry_t), __destroy_entry)) {
            for (size_t j = 0; j < i; ++j)
                ll_free(&arr[j]);
            free(arr);
            return 0;
        }

    h->arr = arr;
    h->key_sz = key_sz;
    h->val_sz = val_sz;
    h->buckets = buckets;
    h->__ctx = (struct __ctx){key_free, val_free};
    h->hash_fn = __hash;
    h->cmp_fn = memcmp;
    return 1;
}

void set_hash(hash_t *h, size_t (*hash)(const void *, size_t)) {
    if (hash)
        h->hash_fn = hash;
}

void set_cmp(hash_t *h, int (*cmp)(const void *, const void *, size_t)) {
    if (cmp)
        h->cmp_fn = cmp;
}

void ht_free(hash_t *h) {
    if (!h)
        return;
    for (size_t i = 0; i < h->buckets; ++i)
        ll_free(&h->arr[i]);
    free(h->arr);

    h->arr = NULL;
    h->key_sz = 0;
    h->val_sz = 0;
    h->buckets = 0;
    h->__ctx = (struct __ctx){NULL, NULL};
}

int ht_put(hash_t *h, const void *key, const void *val) {
    size_t i = h->hash_fn(key, h->key_sz) % h->buckets;
    if (ht_get(h, key, NULL))
        return 0; // key is already occupied
    __entry_t e;
    if (!__create_entry(key, val, h->key_sz, h->val_sz, &e, &h->__ctx))
        return 0;
```

```
    return ll_inserthead(&h->arr[i], &e);
}

int ht_get(const hash_t *h, const void *key, void *out) {
    size_t i = h->hash_fn(key, h->key_sz) % h->buckets;
    linklist_t *l = h->arr + i;
    for (ll_node *cur = l->head; cur; cur = cur->next) {
        __entry_t *e = (__entry_t *)cur->data;
        if (h->cmp_fn(e->key, key, h->key_sz) == 0) {
            if (out)
                memcpy(out, e->val, h->val_sz);
            return 1;
        }
    }
    return 0;
}

int ht_rem(hash_t *h, const void *key) {
    size_t i = h->hash_fn(key, h->key_sz) % h->buckets;
    linklist_t *l = h->arr + i;
    int j = 0;
    for (ll_node *cur = l->head; cur; cur = cur->next, j++) {
        __entry_t *e = (__entry_t *)cur->data;
        if (h->cmp_fn(e->key, key, h->key_sz) == 0) {
            __entry_t removed;
            return ll_delete(l, j, &removed);
        }
    }
    return 0;
}

#endif // HASH_IMPLEMENTATION
#endif // HASH_H
```

Library — set.h (Hash set)

```
// set.h

#ifndef SET_H
#define SET_H

#include <stddef.h> // size_t
typedef struct hash_t set_t;

int set_init(set_t *s, size_t elem_sz, void (*free_ele)(void *));
// tip: use set_hash set_cmp for more complex datatypes
int set_insert(set_t *s, const void *elem);
int set_contains(const set_t *s, const void *elem);
int set_remove(set_t *s, const void *elem);
void set_free(set_t *s);

#ifdef SET_IMPLEMENTATION

#ifndef HASH_IMPLEMENTATION
#define HASH_IMPLEMENTATION
#endif // HASH_IMPLEMENTATION
#include "hash.h"

const char val = 'y';

int set_init(set_t *s, size_t elem_sz, void (*free_ele)(void *)) {
    return ht_init(s, elem_sz, sizeof(char), 30, free_ele, NULL);
}

int set_insert(set_t *s, const void *elem) {
    return ht_put(s, elem, &val);
}

int set_contains(const set_t *s, const void *elem) {
    return ht_get(s, elem, NULL);
}

int set_remove(set_t *s, const void *elem) {
    return ht_rem(s, elem);
}

void set_free(set_t *s) {
    ht_free(s);
}

#endif // SET_IMPLEMENTATION
#endif // SET_H
```

Library — sort.h (Sorting algorithms)

```c
// sort.h
#ifndef SORT_H
#define SORT_H

#include <stddef.h>

void insertion_sort(void *base, size_t nmemb, size_t size,
                    int (*cmp)(const void *, const void *));
void selection_sort(void *base, size_t nmemb, size_t size,
                    int (*cmp)(const void *, const void *));
void merge_sort(void *base, size_t nmemb, size_t size,
                int (*cmp)(const void *, const void *));
void bubble_sort(void *base, size_t nmemb, size_t size,
                 int (*cmp)(const void *, const void *));
void quick_sort(void *base, size_t nmemb, size_t size,
                int (*cmp)(const void *, const void *));
#endif // SORT_H

#include <string.h>

#define ELE_ADDR(arr, sz, idx)                                                 \
    ((void *)((unsigned char *)(arr) + ((size_t)(idx) * (size_t)(sz))))

static void bswap(void *a, void *b, size_t sz) {
    unsigned char tmp[sz];
    memcpy(tmp, a, sz);
    memcpy(a, b, sz);
    memcpy(b, tmp, sz);
}

#ifdef INSERT_SORT_IMPL

static size_t bsearch_left(void *base, size_t nmemb, size_t size,
                           int (*cmp)(const void *, const void *),
                           const void *key) {
    size_t l, r, m;
    l = 0;
    r = nmemb;
    while (l < r) {
        m = (l + r) / 2;
        if (cmp(ELE_ADDR(base, size, m), key) < 0)
            l = m + 1;
        else
            r = m;
    }
    return l;
}

void insertion_sort(void *base, size_t nmemb, size_t size,
                    int (*cmp)(const void *, const void *)) {
    if (!base || !cmp || nmemb < 2 || size == 0)
        return;
    unsigned char tmp[size];

    size_t sorted = 1;

    while (sorted != nmemb) {
        size_t i = bsearch_left(
            base, sorted, size, cmp,
            ELE_ADDR(base, size, sorted)); // Search in the sorted subsequence
        memcpy(tmp, ELE_ADDR(base, size, sorted), size);
        memmove(ELE_ADDR(base, size, i + 1), ELE_ADDR(base, size, i),
                (sorted - i) * size);
        memcpy(ELE_ADDR(base, size, i), tmp, size);
        sorted++;
    }
}

#endif // INSERT_SORT_IMPL

#ifdef SELECT_SORT_IMPL

void selection_sort(void *base, size_t nmemb, size_t size,
                    int (*cmp)(const void *, const void *)) {
    if (!base || !cmp || size == 0 || nmemb < 2)
        return;
```

```
    for (size_t i = 0; i < nmemb - 1; ++i) {
        size_t min_idx = i;
        for (size_t j = i + 1; j < nmemb; ++j)
            if (cmp(ELE_ADDR(base, size, j), ELE_ADDR(base, size, min_idx)) < 0)
                min_idx = j;
        // swap arr[i] and arr[min_idx]
        bswap(ELE_ADDR(base, size, i), ELE_ADDR(base, size, min_idx), size);
    }
}

#endif // SELECT_SORT_IMPL

#ifdef MERGE_SORT_IMPL

static void merge_arr(void *base, size_t size, size_t left, size_t middle,
                      size_t right, int (*cmp)(const void *, const void *)) {
    size_t m, n;
    m = middle - left;
    n = right - middle;
    unsigned char left_part[m * size];
    unsigned char right_part[n * size];

    memcpy(left_part, ELE_ADDR(base, size, left), m * size);
    memcpy(right_part, ELE_ADDR(base, size, middle), n * size);

    size_t i, j, k;
    i = 0;
    j = 0;
    k = left;

    while (i < m && j < n) {
        if (cmp(ELE_ADDR(left_part, size, i), ELE_ADDR(right_part, size, j)) <=
            0) {
            memcpy(ELE_ADDR(base, size, k), ELE_ADDR(left_part, size, i), size);
            ++i;
        } else {
            memcpy(ELE_ADDR(base, size, k), ELE_ADDR(right_part, size, j),
                   size);
            ++j;
        }
        ++k;
    }

    while (i < m) {
        memcpy(ELE_ADDR(base, size, k), ELE_ADDR(left_part, size, i), size);
        ++i;
        ++k;
    }

    while (j < n) {
        memcpy(ELE_ADDR(base, size, k), ELE_ADDR(right_part, size, j), size);
        ++j;
        ++k;
    }
}

void merge_sort(void *base, size_t nmemb, size_t size,
                int (*cmp)(const void *, const void *)) {
    if (!base || !cmp || size == 0)
        return;
    size_t w, l, r, m;
    w = 1;

    while (w < nmemb) {
        l = 0;

        while (l < nmemb) {
            m = (l + w) < nmemb ? (l + w) : nmemb; // min(l + w, nmemb)
            r = (l + 2 * w) < nmemb ? (l + 2 * w)
                                    : nmemb; // min(l + 2 * w, nmemb)

            merge_arr(base, size, l, m, r, cmp);

            l += 2 * w;
        }
        w <<= 1;
    }
}

#endif // MERGE_SORT_IMPL
```

```
#ifdef BUBBLE_SORT_IMPL

void bubble_sort(void *base, size_t nmemb, size_t size,
                 int (*cmp)(const void *, const void *)) {
    if (!base || !cmp || size == 0)
        return;

    for (size_t i = 0; i < nmemb; ++i) {
        int swapped = 0;

        for (size_t j = 0; j < nmemb - i - 1; ++j) {
            if (cmp(ELE_ADDR(base, size, j), ELE_ADDR(base, size, j + 1)) > 0) {
                bswap(ELE_ADDR(base, size, j), ELE_ADDR(base, size, j + 1),
                      size);
                swapped = 1;
            }
        }

        if (!swapped)
            break;
    }
}

#endif // BUBBLE_SORT_IMPL

#ifdef QUICK_SORT_IMPL

static size_t partition(void *base, size_t size, size_t low, size_t high,
                        int (*cmp)(const void *, const void *)) {
    void *pivot = ELE_ADDR(base, size, high);
    size_t i = low;

    for (size_t j = low; j < high; ++j) {
        if (cmp(ELE_ADDR(base, size, j), pivot) <= 0) {
            bswap(ELE_ADDR(base, size, i), ELE_ADDR(base, size, j), size);
            ++i;
        }
    }

    bswap(ELE_ADDR(base, size, i), ELE_ADDR(base, size, high), size);

    return i;
}

static void __qsort_rec(void *base, size_t size, size_t low, size_t high,
                        int (*cmp)(const void *, const void *)) {
    if (low >= high)
        return;
    size_t p_idx = partition(base, size, low, high, cmp);
    if (p_idx > low)
        __qsort_rec(base, size, low, p_idx - 1, cmp);
    if (p_idx < high)
        __qsort_rec(base, size, p_idx + 1, high, cmp);
}

void quick_sort(void *base, size_t nmemb, size_t size,
                int (*cmp)(const void *, const void *)) {
    if (!base || !cmp || size == 0 || nmemb < 2)
        return;
    __qsort_rec(base, size, 0, nmemb - 1, cmp);
}

#endif // QUICK_SORT_IMPL
#undef ELE_ADDR
```

Program 1 — Book Allocation Problem

Question

These days, many students show little interest in reading textbooks. Instead, they often rely on AI tools to develop superficial or vague concepts, whereas textbooks provide a more comprehensive and structured understanding of various topics. Assume I recommend some excellent books on Data structures and algorithms and ask you to read them in a collaborative way – each student reads only a subset of the book pool, to help maintain focus (especially for beginners). Help me allocate books such that the maximum number of pages assigned to any student is minimized.
You are given an integer s, the number of students, and a sorted array pages such that pages[i] denotes the number of pages in book i. Assume there are n books.

Sample run
Input:
pages = [10, 20, 30, 40, 50, 60]
s = 4

Output:
Student 1: [10, 20, 30] -> 60 pages
Student 2: [40] -> 40 pages
Student 3: [50] -> 50 pages
Student 4: [60] -> 60 pages

Data Structures / Algorithm Used

- **Data structure:** a plain integer array arr[n] holding the page counts, declared using a variable-length array.
- **Algorithm:** Binary search on the answer. The minimum possible value of the "maximum pages assigned to one student" lies between max(books) and sum(books). For any candidate value m in this range, a greedy feasibility check (is_feasible) determines whether the books can be split among s students so that no student gets more than m pages. Binary search narrows this range to find the smallest feasible m, after which the same greedy partitioning is replayed once more to print the actual allocation.

Program 1 — Code

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

bool is_feasible(const int *books, int n, int s, int max) {
    // Returns if it is possibile to distribute n books to s children such that pages alloted for
each children <=max
    int curr = 0;
    for (int i = 0; i < n; ++i) {
        if (curr + books[i] > max) {
            s--;
            if (!s) return false;
            curr = books[i];
        } else {
            curr += books[i];
        }
    }
    return true;
}

int main() {
    int n, s, c, max, sum;
    printf("#Books: ");
    scanf("%d", &n);
    int arr[n];
    max = INT_MIN;
    sum = 0;
    for (int i = 0; i < n; ++i) {
        printf("Pages in book-%d: ", i + 1);
        scanf("%d", &c);
        arr[i] = c;
        max = c > max ? c : max;
        sum += c;
    }
    printf("#Students: ");
    scanf("%d", &s);

    // Binary search: the minimum max_pages lies between max(books) and sum(books)
    // All value >= the minimum return true on feasibility check and the rest don't
    int l, r, m;
    l = max;
    r = sum;
    while (l < r) {
        m = (l + r) / 2;

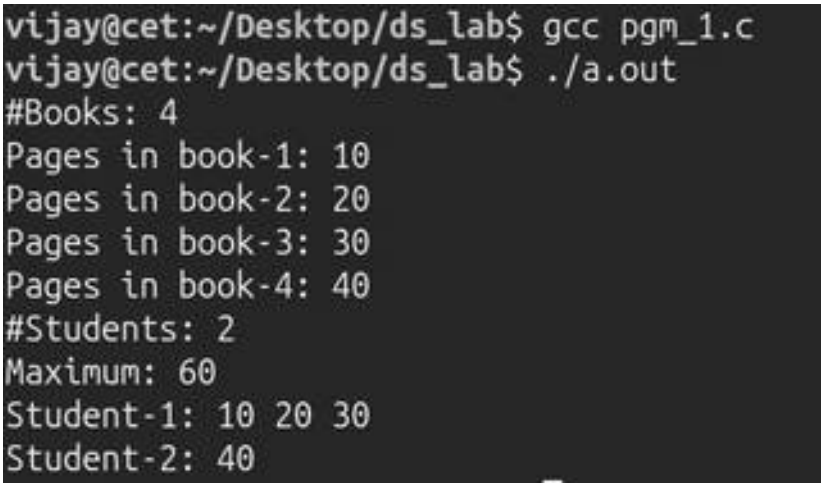
        if (is_feasible(arr, n, s, m)) r = m;
        else l = m + 1;

    }

    // Our answer is in m
    // Reconstruct the book allocation
    printf("Maximum: %d\n", m);
    int curr = 0;
    int st = 1;
    printf("Student-1: ");
    for (int i = 0; i < n; ++i) {
        if (curr + arr[i] > m) {
            st++;
            printf("\nStudent-%d: %d ", st, arr[i]);
            curr = arr[i];
        } else {
            printf("%d ", arr[i]);
            curr += arr[i];
        }
    }
    printf("\n");

    return 0;
}
```

Program 1 — Output



Program 2 — NTRU Polynomial Addition (Nth Degree Truncated Polynomial Ring Unit)

Question

Cryptography is the science of securing confidential messages exchanged between two entities by encoding them to prevent unauthorized access. More often, the messages are represented as polynomials (forget about how this is actually done) for easy encoding and in some modern cryptosystems, to make the system efficient, these polynomials are often sparse. Consider a simplified version of such a system NTRU (Nth degree Truncated polynomial Ring Unit), wherein the polynomial coefficients are in $\mathbb{Z}_3$, i.e., only allowed values are 0, 1, 2 and the polynomial degree is at most 6. For encoding a message m, first a polynomial **`m(x)`** *is constructed. The encoded message* **`c(x)`** *called cipher is found as*

`c(x) = m(x) + k(x)`

where k(x) is called the private key polynomial. Operations are done like normal addition, but modulo 3. Your task now is to input m(x), k(x) and compute c(x).

`Sample run`
`Input:`
`k(x) = x^5 + x^2 + 1      m(x) = x^6 + x^2 + 2`

`Output:`
`c(x) = x^6 + x^5 + 2x^2`

Data Structures / Algorithm Used

- **Data structure:** a fixed-size array of 7 integers (***`coeff[7]`***) wrapped inside a ***`struct ntru`*** (aliased as ***`poly_t)`***, representing coefficients for degrees 0 through 6 of a truncated polynomial over $\mathbb{Z}_3$.
- **Algorithm:** straightforward element-wise (index-wise) array addition of the two coefficient arrays, followed by a mod 3 reduction (***`(a[i] + b[i]) % 3`***) applied to every coefficient — this directly implements polynomial addition over the ring $\mathbb{Z}_3$.
- Input coefficients are also individually reduced mod 3 at read time so that any raw input is normalized into the valid ***`{0, 1, 2}`*** range.

Program 2 — Code

```
#include <stdio.h>

// Nth degree truncated polynomial unit
struct ntru {
    int coeff[7];   // Max degree is 6
};

typedef struct ntru poly_t;

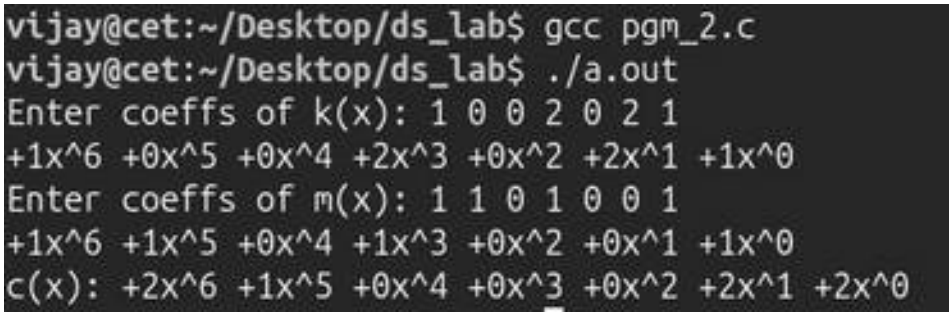
void poly_read(poly_t *p) {
    // Accept input as sequence of doubles from degree 6 to 0
    for (int i = 6; i >= 0; --i)
        scanf("%d", &p->coeff[i]);
    getc(stdin);
    // Convert to mod 3
    for (int i = 0; i < 7; ++i)
        p->coeff[i] = p->coeff[i] % 3;
}

poly_t poly_add(poly_t a, poly_t b) {
    poly_t c;
    for (int i = 0; i < 7; ++i)
        c.coeff[i] = (a.coeff[i] + b.coeff[i]) % 3;
    return c;
}

void poly_print(poly_t p) {
    for (int i = 6; i >= 0; --i)
        printf("%+dx^%d ", p.coeff[i], i);
    printf("\n");
}

int main() {
    poly_t k, m, c;
    printf("Enter coeffs of k(x): ");
    poly_read(&k);
    poly_print(k);
    printf("Enter coeffs of m(x): ");
    poly_read(&m);
    poly_print(m);
    printf("c(x): ");
    c = poly_add(k, m);
    poly_print(c);
    return 0;
}
```

Program 2 — Output



Program 3 — Sparse Matrix Addition and Transpose

Question

You have been hired for developing animation games by RoboGames Ltd. The game environment has multiple layers of objects as follows:
Layer 1: Static buildings
Layer 2: Moving vehicles
Layer 3: Animated characters
Each object has a position represented on a 2D grid. Since most positions are empty (no object), sparse matrices are employed to represent updates from different layers of a virtual scene. The individual matrices are added to compute the total transformation for the frame. The final transformation matrix is transposed before sending to the GPU (Graphical Processing Unit) for rendering. This is to match the column-major format expected by shaders. (Shaders are small programs that run on a GPU to determine how objects and scenes are rendered on the screen). Your job now is to input the updates from a scene and feed them to the shader.

Data Structures / Algorithm Used

- **Data structure:** Sparse matrix — triplet (array of tuples) representation. Each non-zero cell is stored as a ***`struct tuple { i, j, v }`*** (row, column, value) in a dynamically allocated array, inside a ***`struct sparse_matrix { n, r, c, tuple nonzeroes[] }`***.
- **Algorithms used:**
- **get_elem** — linear search through the triplet array to fetch the value at a given (i, j), returning 0 for any position not explicitly stored (i.e., an implicit zero).
- **mat_add** — computes the sum of two sparse matrices of equal dimensions by iterating over every ***`(i, j)`*** cell of the dense index space, summing the corresponding elements from both matrices via get_elem, and storing only the non-zero results as new triplets.
- **mat_trans** — computes the transpose of a sparse matrix in ***`O(n)`*** by simply swapping the row and column index of every stored triplet.
- **destroy_mat** — frees the dynamically allocated triplet array.
- The main function chains these operations: ***`t = a + b`***, ***`d = t + c`***, then ***`t = transpose(d)`***, matching the "sum three layers, then transpose for the GPU" requirement.

Program 3 — Code

```
#include <stdio.h>
#include <stdlib.h> // calloc, free
#include <string.h> // memset
#include <assert.h>
struct tuple {
    int i;  // row
    int j;  // column
    int v;  // value
};
struct sparse_matrix {
    int n;       // Number of non-zero tuples
    int r;       // Number of rows
    int c;       // Number of cols
    struct tuple *nonzeroes;
};
typedef struct sparse_matrix spmat_t;
void read_mat(spmat_t *sp) {
    printf("#Rows: ");
    scanf("%d", &sp->r);
    printf("#Column: ");
    scanf("%d", &sp->c);
    int n;
    printf("#Nonzeroes: ");
    scanf("%d", &n);
    sp->n = n;
    sp->nonzeroes = (struct tuple *)calloc(n, sizeof(struct tuple));
    while (n--) {
        printf(">>");
        struct tuple *c = &sp->nonzeroes[n];
        scanf("%d@%d,%d", &c->v, &c->i, &c->j);
    }
}
void destroy_mat(spmat_t *sp) {
    if (sp->nonzeroes) free(sp->nonzeroes);
    memset(sp, 0, sizeof(spmat_t));
}
int get_elem(spmat_t sp, int i, int j) {
    for (int k = 0; k < sp.n; ++k)
        if (sp.nonzeroes[k].i == i && sp.nonzeroes[k].j == j)
            return sp.nonzeroes[k].v;
    return 0;
}
void print_mat(spmat_t sp) {
    for (int i = 0; i < sp.r; ++i) {
        for (int j = 0; j < sp.c; ++j)
            printf("%d ", get_elem(sp, i, j));
        printf("\n");
    }
}

void mat_add(spmat_t a, spmat_t b, spmat_t *c) {
    assert(a.r == b.r && a.c == b.c);
    c->r = a.r;
    c->c = a.c;
    int cnt = 0;
    int v;
    c->nonzeroes = (struct tuple *)calloc(a.n + b.n, sizeof(struct tuple));
    for (int i = 0; i < a.r; ++i)
        for (int j = 0; j < a.c; ++j)
            if ( (v = get_elem(a, i, j) + get_elem(b, i, j)) != 0)
                c->nonzeroes[cnt++] = (struct tuple) {
                        .i = i,
                        .j = j,
                        .v = v
                };
    c->n = cnt;
}

void mat_trans(spmat_t a, spmat_t *t) {
    t->r = a.c;
    t->c = a.r;
    t->n = a.n;
    t->nonzeroes = (struct tuple*)calloc(a.n, sizeof(struct tuple));
    for (int i = 0; i < a.n; ++i) {
        struct tuple n = a.nonzeroes[i];
        t->nonzeroes[i] = (struct tuple) {
                .i = n.j,
```

```
                .j = n.i,
                .v = n.v
        };
    }
}

int main() {
    spmat_t a, b, c, t, d;
    printf("Static Building: \n");
    read_mat(&a);
    print_mat(a);

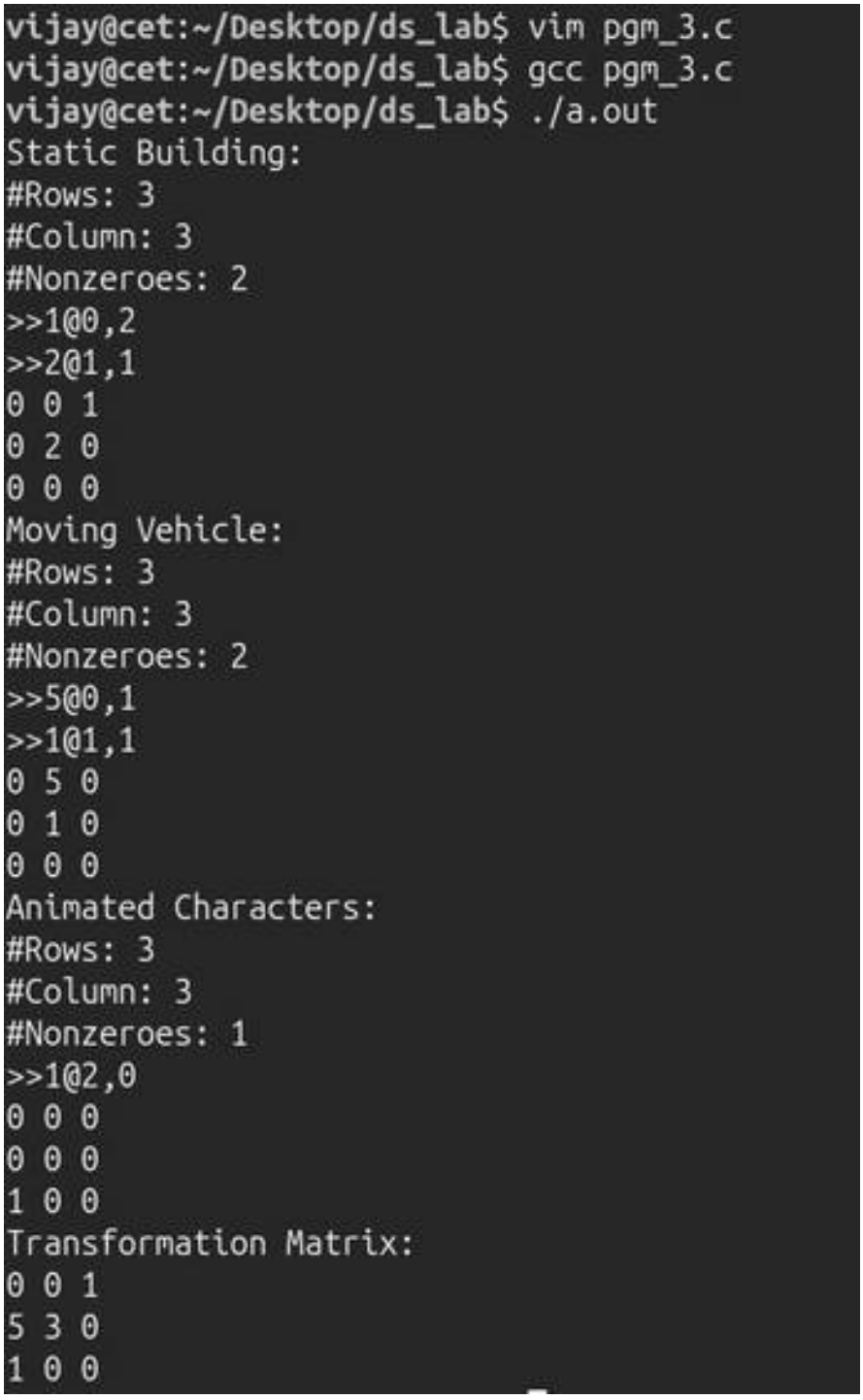
    printf("Moving Vehicle: \n");
    read_mat(&b);
    print_mat(b);

    printf("Animated Characters: \n");
    read_mat(&c);
    print_mat(c);

    printf("Transformation Matrix: \n");
    mat_add(a, b, &t);
    mat_add(t, c, &d);
    destroy_mat(&t); // free up t before reuse
    mat_trans(d, &t);
    print_mat(t);

    destroy_mat(&a);
    destroy_mat(&b);
    destroy_mat(&c);
    destroy_mat(&t);
    destroy_mat(&d);
    return 0;
}
```

Program 3 — Output



Program 4 — Infix to Postfix Conversion and Evaluation

Question

Write a menu-driven program for the following operations:
(a) Convert an infix expression to a postfix expression.
(b) Evaluate the postfix expression.

Data Structures / Algorithm Used

- **Data structure:** a stack is used twice — once holding char operators during infix→postfix conversion, and once holding double operands during postfix evaluation.
- **Algorithms used:**
- **Infix-to-Postfix conversion** (operator-precedence): digits are appended directly to the output; `'('` is pushed onto the operator stack; on `')'`, operators are popped and appended to the output until the matching `'('` is found and discarded; for any other operator, operators of higher-or-equal precedence are popped from the stack to the output before the current operator is pushed. Precedence is decided by the helper `precedence()` (`+ - = 1, * / = 2, ^ = 3`). Any operators remaining on the stack at the end are appended to the output.
- **Postfix evaluation:** the postfix string is scanned left to right; digits are pushed onto an operand stack; on encountering an operator, the top two operands are popped, the operator is applied, and the result is pushed back. The final value left on the stack is the result.

Program 4 — Code

```
#include <stdio.h>
#include <ctype.h>
#include <math.h>
#define STACK_IMPLEMENTATION
#include "stack.h"

#define BUFFER_LEN 64

int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return 0; // shouldn't be reached
    }
}

void itopfix(const char infix[BUFFER_LEN], char res[BUFFER_LEN]) {
    stack_t cstck;  // stack of operator
    int i = 0;      // current string index
    stack_init(&cstck, sizeof(char), NULL);

    for (int j = 0; infix[j] != '\0';  ++j) {
        char c = infix[j];
        if (isspace(c)) continue;

        if (isdigit(c)) res[i++] = c;

        else if (c == '(') stack_push(&cstck, &c);

        else if (c == ')') {
            char d;
            stack_peek(&cstck, &d);
            while (d != '(') {
                stack_pop(&cstck, &d);
                res[i++] = d;
                stack_peek(&cstck, &d);
            }
            stack_pop(&cstck, &d); // pop '('
        }
        // Assume invalid characters will not be entered
        // Assume no variables
        else {
            char d;
            if (!stack_isempty(&cstck)) {
                stack_peek(&cstck, &d);
                while (precedence(d) >= precedence(c)) {
                    stack_pop(&cstck, &d);
                    res[i++] = d;
                    if (stack_isempty(&cstck)) break;
                    stack_peek(&cstck, &d);
                }
            }
            stack_push(&cstck, &c);
        }
    }
    while (!stack_isempty(&cstck)) {
        char d;
        stack_pop(&cstck, &d);
        res[i++] = d;
    }
    res[i] = '\0'; // terminate the string
    stack_free(&cstck);
}

int ctoi(char c) {
    return c - '0';
}

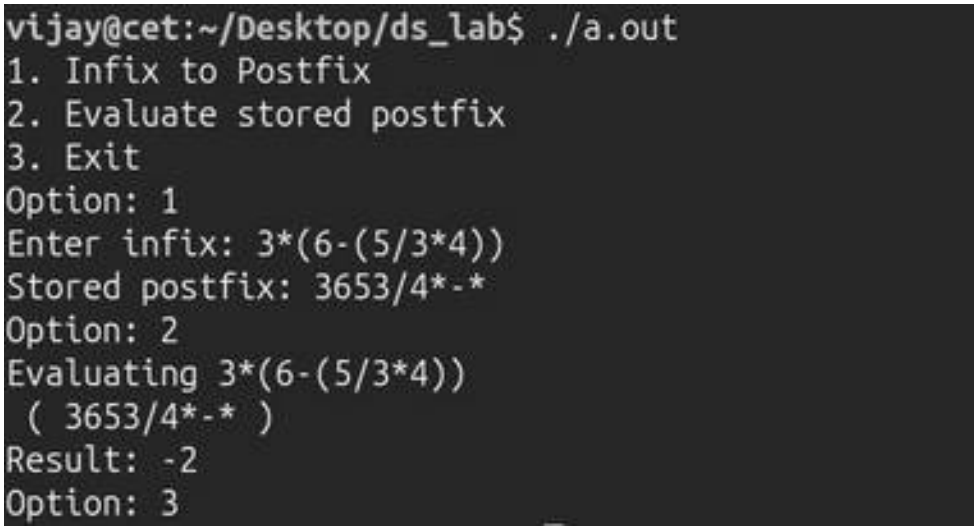
double peval(char postfix[BUFFER_LEN]) {
    stack_t dstck;
```

```
    stack_init(&dstck, sizeof(double), NULL);

    for (int i = 0; postfix[i] != '\0'; ++i) {
        char c = postfix[i];
        if (isdigit(c)) {
            double i = (double)ctoi(c);    // Assume single digit integers
            stack_push(&dstck, &i);
        }
        // Operator otherwise
        else {
            // Only binary operators supported
            double a, b, d;
            stack_pop(&dstck, &b);
            stack_pop(&dstck, &a);
            switch (c) {
                case '+':
                    d = a + b;
                    break;
                case '-':
                    d = a - b;
                    break;
                case '*':
                    d = a * b;
                    break;
                case '/':
                    d = a / b;
                    break;
                case '^':
                    d = pow(a, b);
                    break;
                default:
                    d = -1; // shouldn't reach here
                    break;
            }
            stack_push(&dstck, &d);
        }
    }
    double r;
    stack_pop(&dstck, &r);
    stack_free(&dstck);
    return r;
}

int main() {
    char infix[BUFFER_LEN] = {0};
    char postfix[BUFFER_LEN] = {0};
    int c, r;
    r = 1;
    printf( "1. Infix to Postfix\n"
            "2. Evaluate stored postfix\n"
            "3. Exit\n"
            );
    do {
        printf("Option: ");
        scanf("%d", &c);
        getc(stdin);
        switch (c) {
            case 1:
                printf("Enter infix: ");
                fgets(infix, BUFFER_LEN - 1, stdin);
                itopfix(infix, postfix);
                printf("Stored postfix: %s\n", postfix);
                break;
            case 2:
                printf("Evaluating %s ( %s )\n", infix, postfix);
                printf("Result: %g\n", peval(postfix));
                break;
            case 3:
                r = 0;
                break;
            default:
                printf("Invalid option\n");
                break;
        }
    } while (r);
    return 0;
}
```

Program 4 — Output



Program 5 — Printer Queue Simulation

Question

In a faculty room, several teachers share a single WiFi-enabled printer. Each teacher sends PDF documents to the printer which are placed into the printer spool where the documents wait to be printed in the order they arrive. The printer operates at a speed of 30 ppm (pages per minute). Simulate this printing scenario with a menu to display the following:
(a) name of the teacher whose PDF is currently being printed
(b) waiting time for the prof. who submitted the doc
(c) name of the teacher who has to wait the most.

Data Structures / Algorithm Used

- **Data structure:** a queue is used to model the printer spool in strict FIFO order. Each element stored is a ***`struct print_job { teacher, filename, pages }`***.
- **Algorithms used:**
- **Enqueue / Dequeue (FIFO):** new print jobs are added at the tail (enqueue) and the job currently printing is removed from the head (dequeue), matching first-come-first-served printer behaviour.
- **Peek:** viewing the current job without removing it from the queue.
- **Linear traversal** for wait-time computation: the time (in seconds) a job must wait is computed by walking the queue from the head up to (but not including) the target job, summing ***`pages * SECS_PER_PAGE`*** (with ***`SECS_PER_PAGE = 2`***, i.e. the printer's 30 ppm rate) for every job ahead of it in the queue.

Program 5 — Code

```
#include <stdio.h>
#define QUEUE_IMPLEMENTATION
#include "queue.h"

#define MAX_LEN         64
#define SECS_PER_PAGE   2   // 30 ppm

struct print_job {
    char    teacher[MAX_LEN];
    char    filename[MAX_LEN];
    int     pages;
};

typedef struct print_job job_t;

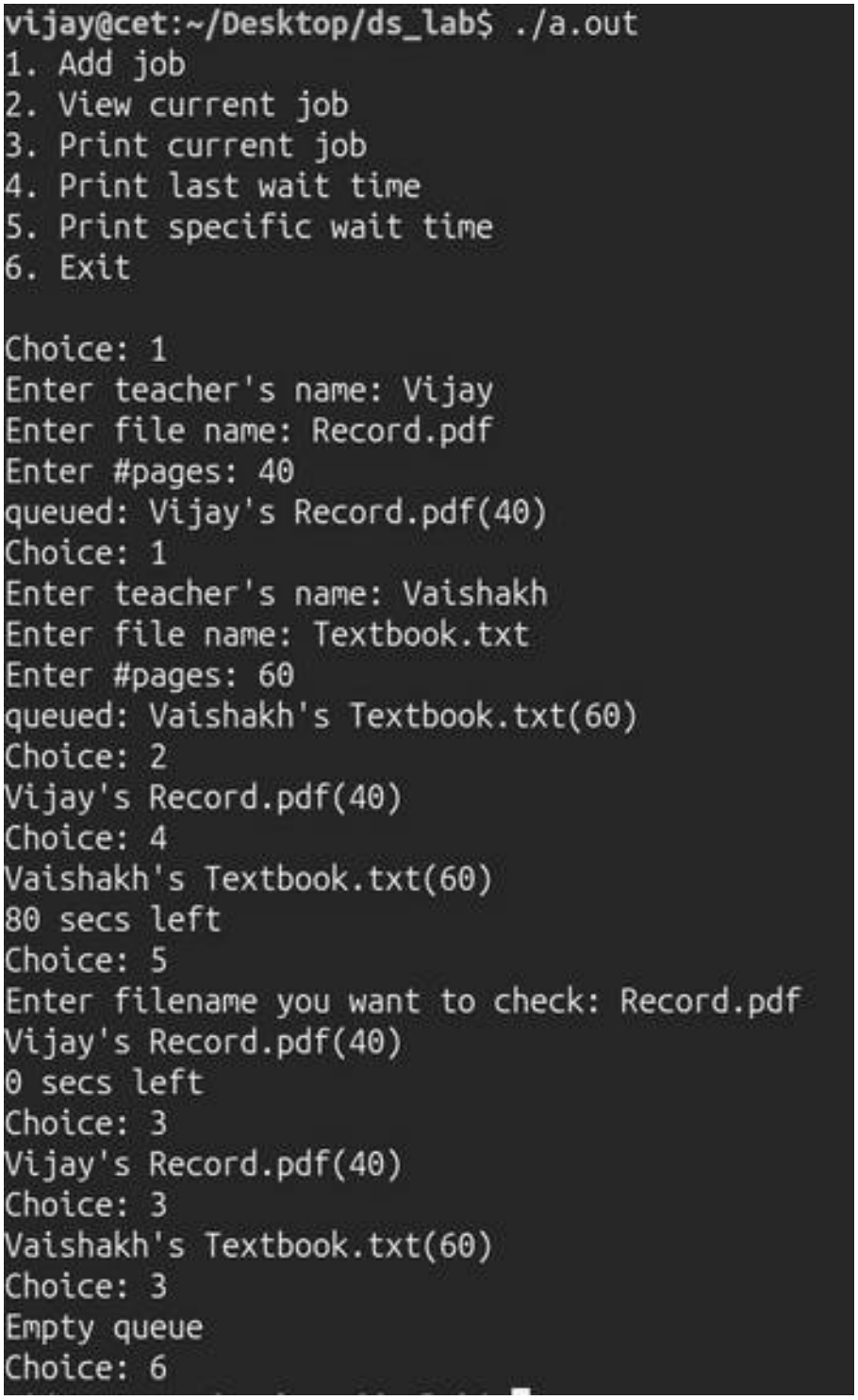
void print_job(job_t j) {
    printf("%s's %s(%d)\n", j.teacher, j.filename, j.pages);
}

int main() {
    queue_t pq; // printer queue
    q_init(&pq, sizeof(job_t), NULL);
    printf( "1. Add job\n"
            "2. View current job\n"
            "3. Print current job\n"
            "4. Print last wait time\n"
            "5. Print specific wait time\n"
            "6. Exit\n\n");
    char tmp[MAX_LEN];
    job_t j;
    int ch, s, r;
    dll_node *cur;
    r = 1;
    do {
        printf("Choice: ");
        scanf("%d", &ch);
        getc(stdin);
        switch (ch) {
            case 1:
                printf("Enter teacher's name: ");
                fgets(j.teacher, MAX_LEN, stdin);
                j.teacher[strcspn(j.teacher, "\n")] = '\0';
                printf("Enter file name: ");
                fgets(j.filename, MAX_LEN, stdin);
                j.filename[strcspn(j.filename, "\n")] = '\0';

                printf("Enter #pages: ");
                scanf("%d", &j.pages);
                getc(stdin);
                enqueue(&pq, &j);
                printf("queued: ");
                print_job(j);
                memset(&j, 0, sizeof(job_t));
                break;
            case 2:
                if (q_isempty(&pq)) printf("Empty queue\n");
                else {
                    peek(&pq, &j);
                    print_job(j);
                }
                break;
            case 3:
                if (q_isempty(&pq)) printf("Empty queue\n");
                else {
                    dequeue(&pq, &j);
                    print_job(j);
                }
                break;
            case 4:
                s = 0;
                cur = pq.head;
                for (; cur && cur->next; cur = cur->next) s += ((job_t *)(cur->data))->pages *
SECS_PER_PAGE;
                print_job(*(job_t *)cur->data);
                printf("%d secs left\n", s);
                break;
            case 5:
```

```
                printf("Enter filename you want to check: ");
                fgets(tmp, MAX_LEN, stdin);
                tmp[strcspn(tmp, "\n")] = '\0';
                cur = pq.head;
                s = 0;
                for (; cur && strcmp(tmp, ((job_t *)(cur->data))->filename); cur = cur->next)
                    s += ((job_t *)(cur->data))->pages * SECS_PER_PAGE;
                print_job(*(job_t *)cur->data);
                printf("%d secs left\n", s);
                break;
            case 6:
                r = 0;
                break;
        }
    } while (r);
    q_free(&pq);
    return 0;
}
```

Program 5 — Output



Program 6 — Railway Ticket Booking Token Queue

Question

In earlier days, online train ticket reservations were less popular. Everyone used to book tickets at railway station reservation counters by filing a form and waiting for their turn in a queue. For every ticket to be booked, you had to fill out separate forms. Indian Railways has a rule that only one form will be accepted from a customer at a time. If a person has to book multiple tickets, he has to book one ticket during his turn, then go to the tail of the queue and wait for his next turn. There is a token machine at the reservation office entrance where the customer is prompted for the number of tickets to be booked. The machine then issues him a token bearing a token number and the number of tickets to be booked. The customers are served in the token number order. A customer needs to take a token only once even if he has multiple tickets to book. Implement this scenario in a program with a menu to display the following:
(a) the current customer being served
(b) number of customers waiting
(c) list all customers in the queue with the number of forms they have.

Data Structures / Algorithm Used

- **Data structure:** a queue holding ***`struct token { token_no, tickets, name }`*** in strict FIFO (token-number) order.
- **Algorithms used:**
- **Request token** (enqueue): a new customer's details are pushed onto the tail of the queue with a monotonically increasing token_no.
- **Fill form** (partial dequeue + re-queue): the head of the queue is removed and served for one ticket; if the customer still has remaining tickets to book, the same customer (with tickets decremented) is re-enqueued at the tail — directly implementing the rule that a customer must go to the back of the line after each single form.
- **Current customer** (peek) and waiting count (***`q_size`***) are simple ***`O(1)/O(n)`*** queries.
- **List all customers**: a linear traversal of the underlying doubly linked list from head to NULL, printing every queued token.

Program 6 — Code

```
#include <stdio.h>
#include <string.h>
#define QUEUE_IMPLEMENTATION
#include "queue.h"

#define MAX_LEN 32

typedef struct token {
    int token_no;
    int tickets;
    char name[MAX_LEN];
} token_t;

void print_token(token_t t) {
    printf("%d. %s(%d)\n", t.token_no, t.name, t.tickets);
}

int main() {
    queue_t q;
    q_init(&q, sizeof(token_t), NULL);

    printf("1. Request token\n"
           "2. Fill form\n"
           "3. Current cust.\n"
           "4. No. of waiting cust.\n"
           "5. List cust.\n"
           "6. Exit\n\n");
    int ch, r, n;
    token_t t = {0};
    n = 1; // token no
    r = 1;
    do {
        printf("Choice: ");
        scanf("%d", &ch);
        getc(stdin);
        switch (ch) {
        case 1:
            t.token_no = n++;

            printf("Enter name: ");
            fgets(t.name, MAX_LEN, stdin);
            t.name[strcspn(t.name, "\n")] = '\0';

            printf("Enter #tickets: ");
            scanf("%d", &t.tickets);

            enqueue(&q, &t);
            printf("Queued: ");
            print_token(t);
            memset(&t, 0, sizeof(token_t));
            break;
        case 2:
            if (q_isempty(&q))
                printf("Empty queue\n");
            else {
                dequeue(&q, &t);
                print_token(t);
                if (--t.tickets)
                    enqueue(&q, &t);
            }
            break;
        case 3:
            if (q_isempty(&q))
                printf("Empty queue\n");
            else {
                peek(&q, &t);
                print_token(t);
            }
            break;
        case 4:
            if (q_isempty(&q))
                printf("Empty queue\n");
            else
                printf("Waiting: %zu\n", q_size(&q));
            break;
        case 5:
            for (dll_node *cur = q.head; cur; cur = cur->next)
```

```
                print_token(*(token_t *)(cur->data));
            break;
        case 6:
            r = 0;
            break;
        default:
            printf("Invalid option\n");
            break;
        }
    } while (r);

    q_free(&q);
    return 0;
}
```

Program 6 — Output

```
vijay@cet:~/Desktop/ds_lab$ ./a.out
1. Request token
2. Fill form
3. Current cust.
4. No. of waiting cust.
5. List cust.
6. Exit

Choice: 1
Enter name: Vijay
Enter #tickets: 1
Queued: 1. Vijay(1)
Choice: 1
Enter name: Vaishakh
Enter #tickets: 3
Queued: 2. Vaishakh(3)
Choice: 1
Enter name: Teresa
Enter #tickets: 2
Queued: 3. Teresa(2)
Choice: 5
1. Vijay(1)
2. Vaishakh(3)
3. Teresa(2)
Choice: 2
1. Vijay(1)
Choice: 2
2. Vaishakh(3)
Choice: 5
3. Teresa(2)
2. Vaishakh(2)
Choice: 3
3. Teresa(2)
Choice: 4
Waiting: 2
Choice: 6
```

Program 7 — Browser Back/Forward Navigation

Question

You follow this sequence (by clicking appropriate hyperlinks or tabs) for your course registration: ***KTU homepage >>> E-Governance Application >>> Course/Semester Exam Registration >>> Register New Courses***
In the midst, you also want to download your grade card for the second semester examination the result of which got published recently. For this, you may have to click on the "Results" tab. Each hyperlink or tab has an associated address or URL that will take you to the pertinent web page when clicked. To navigate back and forth between the web pages already visited, you will be using the '→' and '←' buttons of the browser. Identify what data structure is appropriate for implementing these "Back" and "Forward" buttons. You are supposed to code those button functionalities in terms of the chosen data structure operations. ***[Hint: You may have to store the URLs (https://ktu.edu.in/, https://app.ktu.edu.in/login.htm etc.) in the data structure.]***

Data Structures / Algorithm Used

- **Data structure:** two stacks — undo holding the browsing history up to and including the current page, and redo holding pages that were undone via "Back" and are available for "Forward".
- **Algorithms used:** this is the standard two-stack browser history technique:
- **Goto** (new navigation): push the new URL/path onto undo; clear ***redo*** entirely, since visiting a fresh page invalidates any previously "forward"-able history.
- **Back**: pop the top of undo and push it onto redo.
- **Forward**: pop the top of ***redo*** and push it back onto ***undo***.
- **Print**: since a stack only exposes its top, the history is printed in visiting order by recursively walking the stack's underlying linked list from tail to head.

Program 7 — Code

```
#include <stdio.h>
#include <string.h>
#define STACK_IMPLEMENTATION
#include "stack.h"

#define LEN 32

typedef char url_t[LEN];

void __ll_rev_print(ll_node *n) {
    if (!n)
        return;
    __ll_rev_print(n->next);
    printf("%s", *(url_t *)(n->data));
}
void print_url(stack_t s) {
    __ll_rev_print(s.head);
    printf("\n");
}
int main() {
    stack_t undo, redo; // undo stack holds the actual URL
    stack_init(&undo, sizeof(url_t), NULL);
    stack_init(&redo, sizeof(url_t), NULL);
    int ch, r;
    url_t tmp;
    r = 1;
    printf("1. Goto\n"
           "2. Back\n"
           "3. Forward\n"
           "4. Print\n"
           "5. Exit\n\n");
    do {
        printf("Choice: ");
        scanf("%d", &ch);
        getc(stdin);
        switch (ch) {
        case 1:
            printf("Enter next path (eg: home/): ");
            fgets(tmp, LEN, stdin);
            tmp[strcspn(tmp, "\n")] = '\0';
            stack_push(&undo, tmp);

            stack_free(&redo);
            stack_init(&redo, sizeof(url_t), NULL); // reinit redo
            break;
        case 2:
            if (stack_isempty(&undo))
                printf("Empty URL\n");
            else {
                stack_pop(&undo, tmp);
                stack_push(&redo, tmp);
            }
            break;
        case 3:
            if (stack_isempty(&redo))
                printf("Empty history\n");
            else {
                stack_pop(&redo, tmp);
                stack_push(&undo, tmp);
            }
            break;
        case 4:
            print_url(undo);
            break;
        case 5:
            r = 0;
            break;
        default:
            printf("Invalid option\n");
            break;
        }
    } while (r);

    stack_free(&undo);
    stack_free(&redo);
    return 0;
}
```

Program 7 — Output

```
vijay@cet:~/Desktop/ds_lab$ ./a.out
1. Goto
2. Back
3. Forward
4. Print
5. Exit

Choice: 1
Enter next path (eg: home/): https:/ktu.edu.in/
Choice: 1
Enter next path (eg: home/): home/
Choice: 4
https:/ktu.edu.in/home/
Choice: 2
Choice: 4
https:/ktu.edu.in/
Choice: 3
Choice: 4
https:/ktu.edu.in/home/
Choice: 2
Choice: 1
Enter next path (eg: home/): login.htm
Choice: 4
https:/ktu.edu.in/login.htm
Choice: 3
Empty history
Choice: 5
```

Program 8 — Find and Replace in a Text Document

Question

In GUI based text editors like gedit, the Find and Replace feature allows you to search for all occurrences of a specific word or phrase and optionally replace them with a new one. Your task is to simulate this functionality in a simplified context as follows – The editor holds the text document as a list of words. Given a key term and a replacement word, your program should:
(a) replace each occurrence of term with word
(b) display the number of replacements made
(c) display the updated document.

Data Structures / Algorithm Used

- **Data structure:** a dynamic array storing each word of the document as a fixed-size char[64] element, automatically growing as more words are appended.
- **Algorithms used:**
- **Tokenizing input**: each line read from stdin is split on whitespace using strtok, and every token is added to the list, building an in-memory word-array representation of the "document".
- **Find and Replace** (***`replace_words`***): a single linear scan (***`O(n)`***) over the word list; every element that exactly matches the search term is overwritten in place, and a counter is incremented for each replacement made.
- **Display**: a linear traversal prints every word of the document in order.

Program 8 — Code

```
#include <assert.h>
#include <stdio.h>
#include <string.h>
#define LIST_IMPLEMENTATION
#include "list.h"
#define LINE 1024
#define LEN 64
typedef char word_t[LEN];

void print_words(list_t words) {
    word_t w;
    for (int i = 0; i < list_size(&words); ++i) {
        get(&words, i, &w);
        printf("%s ", w);
    }
    printf("\n");
}

int replace_words(list_t words, word_t old, word_t new) {
    word_t w;
    int n = 0;
    for (int i = 0; i < list_size(&words); ++i) {
        get(&words, i, &w);
        if (!strcmp(w, old))
            set(&words, i, new) && n++;
    }
    return n;
}

int main(int argc, char **argv) {
    assert(argc == 3 && "Use: ./a.out 'old' 'new'\n");
    list_t words;
    list_init(&words, 20, LEN, NULL);
    char line[LINE];

    while (fgets(line, LINE, stdin) != NULL) {
        char *w;
        char delim[] = " \t\n\r";
        w = strtok(line, delim);
        while (w) {
            add(&words, w);
            w = strtok(NULL, delim);
        }
    }
    word_t old, new;
    int n;
    strncpy(old, argv[1], LEN);
    strncpy(new, argv[2], LEN);
    n = replace_words(words, old, new);
    printf("(%d replacements made)\n", n);
    print_words(words);
    list_free(&words);
    return 0;
}
```

Program 8 — Output

```
vijay@cet:~/Desktop/ds_lab$ more test.txt
8. In GUI based text editors like gedit, the Find and Replace feature allows yo
u to search for
all occurrences of a specific word or phrase and optionally replace them with a
 new one. Your
task is to simulate this functionality in a simplified context as follows - The
 editor holds the text
document as a list of words. Given a key term and a replacement word, your prog
ram should:
(a) replace each occurrence of term with word
(b) display the number of replacements made
(c) display the updated document.
vijay@cet:~/Desktop/ds_lab$ cat test.txt | ./a.out "term" "word"
(2 replacements made)
8. In GUI based text editors like gedit, the Find and Replace feature allows you to
 search for all occurrences of a specific word or phrase and optionally replace the
m with a new one. Your task is to simulate this functionality in a simplified conte
xt as follows - The editor holds the text document as a list of words. Given a key
word and a replacement word, your program should: (a) replace each occurrence of wo
rd with word (b) display the number of replacements made (c) display the updated do
cument.
```

Program 9 — Sparse Polynomial Multiplication

Question

In architectural modeling and CAD-based terrain design, curves are often used to represent features such as elevation contours, road profiles. When two such curve segments (say a curved elevation ramp and a sloped road boundary) need to be blended or analyzed for interaction such as overlapping regions or transition zones, their mathematical representations (typically polynomials) are symbolically multiplied. The resulting product captures combined geometric behavior, aiding in tasks like contour merging, surface shaping, or pattern generation. Your task is to input two curve segments given as sparse univariate polynomials (polynomials in single variable x) and compute their composite curve (product).

Data Structures / Algorithm Used

- **Data structure:** a singly linked list of ***`term_t { coeff, exp }`*** nodes, kept sorted by descending exponent at all times.
- **Algorithm used** — sparse polynomial multiplication: for every pair of terms (***`r ∈ a, s ∈ b`***), the product term (***`r.coeff*s.coeff, r.exp+s.exp`***) is computed and merged into the sorted result list c: the list is scanned from the head until either a node with the same exponent is found (its coefficient is simply incremented — like-term collection) or a node with a smaller exponent is found (the new term is inserted just before it, preserving descending-exponent order). This is the classic sparse-polynomial (linked-list, non-zero-terms-only) multiplication algorithm, with overall cost ***`O(n·m·k)`*** where ***`n`***, ***`m`*** are the term counts of the two operands and ***`k`*** is the length of the growing result list.

Program 9 — Code

```c
#include <stdio.h>
#include <string.h>
#define LL_IMPLEMENTATION
#include "linklist.h"

typedef struct {
    double c;
    int e;
} term_t;

void init_poly(linklist_t *l) { ll_init(l, sizeof(term_t), NULL); }
void read_poly(linklist_t *l) {
    while (1) {
        int c;
        printf("Want to continue?(1): ");
        scanf("%d", &c);
        getc(stdin);
        if (c != 1)
            return;
        term_t t;
        scanf("%lf", &t.c);
        scanf("%d", &t.e);
        getc(stdin);
        ll_inserttail(l, &t);
    }
}
void mul_poly(linklist_t *c, linklist_t *a, linklist_t *b) {
    double coeff;
    int exp;
    for (ll_node *i = a->head; i; i = i->next) {
        for (ll_node *j = b->head; j; j = j->next) {
            term_t r = *(term_t *)i->data;
            term_t s = *(term_t *)j->data;
            coeff = r.c * s.c;
            exp = r.e + s.e;

            int found = 0;
            int y = 0;

            for (ll_node *k = c->head; k; k = k->next, y++) {
                term_t *t = (term_t *)k->data;
                if (t->e == exp) {
                    t->c += coeff;
                    found = 1;
                    break;
                } else if (t->e < exp) break;
            }
            if (!found) {
                term_t x = (term_t){.c = coeff, .e = exp};
                ll_insert(c, y, &x);
            }
        }
    }
}
void print_poly(linklist_t *l) {
    for (ll_node *cur = l->head; cur; cur = cur->next) {
        term_t t = *(term_t *)cur->data;
        printf("%+gx%d ", t.c, t.e);
    }
    printf("\n");
}
int main() {
    linklist_t a, b, c;
    init_poly(&a);
    init_poly(&b);
    init_poly(&c);
    read_poly(&a);
    read_poly(&b);
    print_poly(&a);
    print_poly(&b);
    mul_poly(&c, &a, &b);
    print_poly(&c);
    ll_free(&a);
    ll_free(&b);
    ll_free(&c);
    return 0;
}
```

Program 9 — Output

```
vijay@cet:~/Desktop/ds_lab$ gcc -I headers/ pgm/pgm_9.c
vijay@cet:~/Desktop/ds_lab$ ./a.out
Want to continue?(1): 1
1 4
Want to continue?(1): 1
2 2
Want to continue?(1): 1
3 0
Want to continue?(1): 0
Want to continue?(1): 1
4 3
Want to continue?(1): 1
5 1
Want to continue?(1): 1
6 0
Want to continue?(1): 0
+1x4 +2x2 +3x0
+4x3 +5x1 +6x0
+4x7 +13x5 +6x4 +22x3 +12x2 +15x1 +18x0
```

Program 10 — App Cleaner (Most/Least Frequently Used Apps)

Question

App Cleaner is a sort of utility that comes handy to remove "unwanted apps" when your mobile runs out of storage. It basically works by uninstalling those apps that are less frequently used. For this, the utility requires to know which apps are more frequently used and which are less frequently used. Identify a suitable data structure to implement such an utility. Also, come up with an implementation in terms of the chosen data structure operations. Note that when you open an app, it becomes a "more frequently used app".

Data Structures / Algorithm Used

- **Data structure:** a sorted singly linked list used as a simple MFU/LFU list — the head is always the least-frequently-used app and the tail is always the most-frequently-used one.
- **Algorithms used:**
- **Open app**: the list is scanned for the app name; if found, its node is deleted from its current position, frequency incremented and re-inserted such that it is sorted; if not found, it is simply inserted at the head as a brand-new entry with frequency one.
- **Clean**: the head of the list — i.e. the least-frequently-used app — is deleted, directly implementing the "remove the least frequently used app" behaviour the App Cleaner needs.

Program 10 — Code

```
#include <stdio.h>
#include <string.h>
#define LL_IMPLEMENTATION
#include "linklist.h"
#define LEN 64
typedef struct { char app_name[LEN]; int frq; } app_t;

int main() {
	linklist_t freq_used;
	ll_init(&freq_used, sizeof(app_t), NULL);
	int c, f, r, i;
	char tmp[LEN];
	app_t app;
	r = 1;
	printf("1. Open\n"
	       "2. Clean\n"
	       "3. Exit\n\n");
	do {
	    scanf("%d", &c);
	    getc(stdin);
	    switch (c) {
	    case 1:
	        printf("Enter name of app to open: ");
	        fgets(tmp, LEN, stdin);
	        tmp[strcspn(tmp, "\n")] = '\0';
	        f = 0;
	        i = 0;
	        {
	            int del_idx, ist_idx;
	            del_idx = ist_idx = -1;
	            for (ll_node *cur = freq_used.head; cur; cur = cur->next, i++) {
	                app_t curapp = *(app_t *)cur->data;
	                if (!f && !strcmp(tmp, curapp.app_name)) {
	                    app = curapp;
	                    app.frq++;
	                    del_idx = i;
	                    f = 1;
	                    continue;
	                }
	                if (f && app.frq < curapp.frq)
	                    ist_idx = i;
	                    break;
	                }
	            }
	            if (ist_idx != -1)
	                ll_insert(&freq_used, ist_idx, &app);
	            else if (f)
	                ll_inserttail(&freq_used, &app);
	            if (del_idx != -1)
	                ll_delete(&freq_used, del_idx, &app);
	        }
	        if (!f) {
	            strcpy(app.app_name, tmp);
	            app.frq = 1;
	            ll_inserthead(&freq_used, &app);
	        }
	        memset(tmp, 0, LEN);
	        break;
	    case 2:
	        i = ll_deletehead(&freq_used, &app);
	        if (i)
	            printf("Deleted app: %s\n", app.app_name);
	        else
	            printf("Empty\n");
	        break;
	    case 3:
	        r = 0;
	        break;
	    default:
	        printf("Invalid option\n");
	        break;
	    }
	} while (r);
	ll_free(&freq_used);
	return 0;
}
```

Program 10 — Output

```
vijay@cet:~/Desktop/ds_lab$ ./a.out
1. Open
2. Clean
3. Exit

1
Enter name of app to open: w
1
Enter name of app to open: a
1
Enter name of app to open: a
1
Enter name of app to open: a
1
Enter name of app to open: w
2
Deleted app: w
2
Deleted app: a
2
Empty
3
```

Program 11 — Memory Allocation Strategies (First/Best/Worst Fit)

Question

At one instant, the memory map of a 4MB (4000KB) RAM looks as in Figure 1. Processes (P1, P2, etc.) request the operating system for memory and also release the allocated memory after completing execution. A sample execution trace is shown below:

```
P7  requests for 115 KB
P10 requests for 650 KB
P3  completes execution
P1  completes execution
P6  completes execution
P8  requests for 200 KB
P5  completes execution
P2  completes execution
P9  requests for 37 KB
P10 completes execution
P9  completes execution
P4  completes execution
```

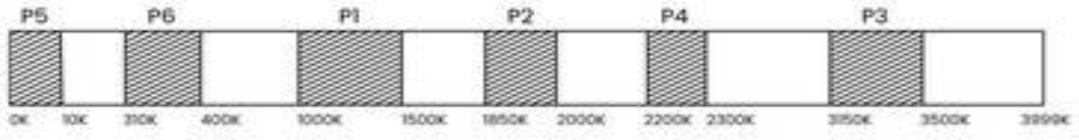


Figure 1

Which strategy among the first fit, best fit, and worst fit performs the best here?

Data Structures / Algorithm Used

- **Data structure:** a singly linked list of ***`chunk_t { name, start, size, alloc }`*** nodes, kept in ascending address order, together modelling the entire 4000 KB memory map as a sequence of free/allocated contiguous chunks.
- **Algorithms used:**
- ***`__alloc`*** locates the chunk containing the target start address and splits it into up to three pieces — an optional leftover-before free chunk, the new allocated chunk, and an optional leftover-after free chunk — replacing the single original list node with these pieces.
- ***`__free`*** marks the named chunk free and, under ***`AUTO_COMPACT`***, coalesces it with any immediately adjacent free chunks (both backward and forward), preventing external fragmentation from lingering.
- **First Fit**: scans from the head and allocates in the first free chunk large enough.
- **Best Fit**: scans all free chunks and allocates in the smallest one that still fits the request.
- **Worst Fit**: scans all free chunks and allocates in the largest one available.
- ***`test_alloc_strat`*** replays the exact sample execution trace from the question against all three strategies in turn so their resulting memory layouts can be compared.

Program 11 — Code

```
#include <assert.h>
#include <stdio.h>
#include <string.h>
#define LL_IMPLEMENTATION
#include "linklist.h"

#define AUTO_COMPACT    // remove to disable auto-compaction on free

typedef struct {
    char name[10];
    int start;
    int size;
    int alloc;
} chunk_t;

void __alloc(linklist_t *mem, char name[10], int start, int size) {
    int i = 0;
    for (ll_node *cur = mem->head; cur; cur = cur->next, i++) {
        chunk_t *ch = (chunk_t *)cur->data;
        int st, sz;
        st = ch->start;
        sz = ch->size;
        if (start < st)
            continue;
        if (start >= st + sz)
            continue;
        assert(!ch->alloc);
        chunk_t prev;
        prev.start = st;
        prev.size = start - st;
        prev.alloc = 0;
        chunk_t new;
        memcpy(new.name, name, 10);
        new.start = start;
        new.size = size;
        new.alloc = 1;
        chunk_t left;
        left.start = start + size;
        left.size = sz - prev.size - new.size;
        left.alloc = 0;
        chunk_t tmp;
        ll_delete(mem, i, &tmp);
        if (prev.size > 0)
            ll_insert(mem, i, &prev);
        else
            i--;
        ll_insert(mem, i + 1, &new);
        if (left.size > 0)
            ll_insert(mem, i + 2, &left);
        break;
    }
#ifdef DEBUG
    printf("Allocated %s: %dKB\n", name, size);
#endif
}

void __free(linklist_t *mem, char name[10]) {
    int i = 0;
    ll_node *cur = mem->head;
    for (; cur; cur = cur->next, ++i) {
        chunk_t *ch = (chunk_t *)cur->data;
        if (ch->alloc && !strcmp(ch->name, name)) {
            ch->alloc = 0;
#ifdef AUTO_COMPACT
            chunk_t prev;
            while (ll_getelem(mem, i - 1, &prev) && !prev.alloc) {
                ch->start = prev.start;
                ch->size += prev.size;
                ll_delete(mem, i - 1, &prev);
                if (!i--)
                    break;
            }
            ll_node *next = cur->next;
            chunk_t nxtch = *(chunk_t *)next->data;
            while (!nxtch.alloc) {
                ch->size += nxtch.size;
                next = next->next;
```

```
                if (!next) {
                    ll_delete(mem, i + 1, &prev);
                    break;
                }
                nxtch = *(chunk_t *)next->data;
                ll_delete(mem, i + 1, &prev);
            }
#endif
#ifdef DEBUG
            printf("Freed: %s\n", name);
#endif
            return;
        }
    }
}

void ff(linklist_t *mem, char name[10], int size) {
    for (ll_node *cur = mem->head; cur; cur = cur->next) {
        chunk_t *ch = (chunk_t *)cur->data;
        if (!ch->alloc && ch->size >= size)
            return __alloc(mem, name, ch->start, size);
    }
    printf("No space!\n");
}

void bf(linklist_t *mem, char name[10], int size) {
    chunk_t *cand = NULL;
    for (ll_node *cur = mem->head; cur; cur = cur->next) {
        chunk_t *ch = (chunk_t *)cur->data;
        if (!ch->alloc && ch->size >= size) {
            if (cand)
                cand = cand->size > ch->size ? ch : cand;
            else
                cand = ch;
        }
    }
    if (cand)
        return __alloc(mem, name, cand->start, size);
    else
        printf("No space!\n");
}

void wf(linklist_t *mem, char name[10], int size) {
    chunk_t *cand = NULL;
    for (ll_node *cur = mem->head; cur; cur = cur->next) {
        chunk_t *ch = (chunk_t *)cur->data;
        if (!ch->alloc && ch->size >= size) {
            if (cand)
                cand = cand->size < ch->size ? ch : cand;
            else
                cand = ch;
        }
    }
    if (cand)
        return __alloc(mem, name, cand->start, size);
    else
        printf("No space!\n");
}

void __print(linklist_t *mem) {
    for (ll_node *cur = mem->head; cur; cur = cur->next) {
        chunk_t ch = *(chunk_t *)cur->data;
        // printf("print\n");
        printf("%s: start: %d, end: %d\n", ch.alloc ? ch.name : "Free",
               ch.start, ch.start + ch.size);
    }
    printf("==================================\n");
}

int main() {
    linklist_t mem;
    ll_init(&mem, sizeof(chunk_t), NULL);
    chunk_t init = (chunk_t){.name = {0}, .start = 0, .size = 4000, .alloc = 0};
    ll_inserthead(&mem, &init);

#ifdef AUTO_COMPACT
    printf("Please note: In this implementation"
           "memory frees initiates auto-compaction.\n\n");
#endif
    int r = 1;
```

```
    int c, s;
    char name[10];
    printf( "1. First fit\n"
            "2. Best fit\n"
            "3. Worst fit\n"
            "4. Free\n"
            "5. Print\n"
            "6. Exit\n\n"
          );
    do {
        printf("Choice: ");
        scanf("%d", &c);
        getc(stdin);
        switch (c) {
        case 1:
            printf("Enter program name: ");
            fgets(name, 10, stdin);
            name[strcspn(name, "\n")] = '\0';
            printf("Enter size: ");
            scanf("%d", &s);
            getc(stdin);
            ff(&mem, name, s);
            break;
        case 2:
            printf("Enter program name: ");
            fgets(name, 10, stdin);
            name[strcspn(name, "\n")] = '\0';
            printf("Enter size: ");
            scanf("%d", &s);
            getc(stdin);
            bf(&mem, name, s);
            break;
        case 3:
            printf("Enter program name: ");
            fgets(name, 10, stdin);
            name[strcspn(name, "\n")] = '\0';
            printf("Enter size: ");
            scanf("%d", &s);
            getc(stdin);
            wf(&mem, name, s);
            break;
        case 4:
            printf("Enter program name: ");
            fgets(name, 10, stdin);
            name[strcspn(name, "\n")] = '\0';
            __free(&mem, name);
            break;
        case 5:
            __print(&mem);
            break;
        case 6:
            r = 0;
            break;
        default:
            printf("Invalid option\n");
            break;
        }
    } while (r);

    ll_free(&mem);
    return 0;
}
```

Program 11 — Output

```
vijay@cet:~/Desktop/ds_lab$ ./a.out
Please note: In this implementationmemory frees initiates auto-compaction.

1. First fit
2. Best fit
3. Worst fit
4. Free
5. Print
6. Exit

Choice: 1
Enter program name: P1
Enter size: 500
Choice: 5
P1: start: 0, end: 500
Free: start: 500, end: 4000
==================================
Choice: 2
Enter program name: P2
Enter size: 1000
Choice: 5
P1: start: 0, end: 500
P2: start: 500, end: 1500
Free: start: 1500, end: 4000
==================================
Choice: 3
Enter program name: P3
Enter size: 2000
Choice: 5
P1: start: 0, end: 500
P2: start: 500, end: 1500
P3: start: 1500, end: 3500
Free: start: 3500, end: 4000
==================================
Choice: 4
Enter program name: P2
Choice: 5
P1: start: 0, end: 500
Free: start: 500, end: 1500
P3: start: 1500, end: 3500
Free: start: 3500, end: 4000
==================================
Choice: 2
Enter program name: P4
Enter size: 500
Choice: 5
P1: start: 0, end: 500
Free: start: 500, end: 1500
P3: start: 1500, end: 3500
P4: start: 3500, end: 4000
==================================
Choice: 6
```

Program 12 — Expression Tree Construction

Question

An expression tree is a tree that represents an arithmetic expression. The leaves of the tree are operands such as constants or variables whereas the internal nodes are all operators. The children of an operator node are its operands. Your job now is to input an arithmetic expression and then

(a) construct an expression tree for the input. [algorithm given below]
(b) display the input expression's prefix and postfix notations. [Hint: think in terms of traversals!]
You may assume that the expression has only binary operands.
Construct-Expression-Tree is the algorithm for constructing a tree for the given expression. It calls an auxiliary procedure Build-Subtree to create a subtree. Two stacks viz. an operatorStack to hold the operators and a nodeStack to hold the tree nodes are used by the construction procedure.

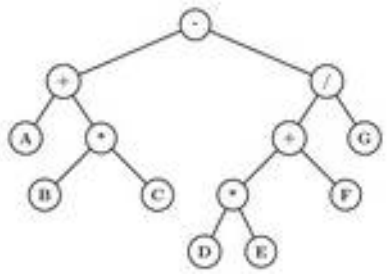


Figure 2

```
Build-Subtree(operatorStack, nodeStack)
1  root = Pop(operatorStack)
2  rightChild = Pop(nodeStack)
3  leftChild = Pop(nodeStack)
4  root.left = leftChild
5  root.right = rightChild
6  Push(nodeStack, root)

Construct-Expression-Tree(expression)
1  operatorStack = a new empty stack
2  nodeStack = a new empty stack
3  for each char in expression
4      if IsOperand(char)
5          newNode = Create-Leaf-Node(char)
6          Push(nodeStack, newNode)
7      elseif char == '('
8          Push(operatorStack, char)
9      elseif char == ')'
10         while Top(operatorStack) != '('
11             Build-Subtree(operatorStack, nodeStack)
12         Pop(operatorStack)   // Discard the matching '('
13     else   // an operator
14         while (! IsEmpty(operatorStack) and Top(operatorStack) != '(' and
                   Precedence(Top(operatorStack)) >= Precedence(char))
15             Build-Subtree(operatorStack, nodeStack)
16         Push(operatorStack, char)
17 while ! IsEmpty(operatorStack)
18     Build-Subtree(operatorStack, nodeStack)
19 return Pop(nodeStack)   // The root of the completed tree
```

Data Structures / Algorithm Used

- **Data structure:** two stacks in sequence — (1) an operator stack, and (2) a stack of tree-node pointers used while building the expression tree from the resulting postfix string.
- **Algorithm used** — ***Construct-Expression-Tree*** / ***Build-Subtree*** (exactly as specified in the lab sheet, expressed here in terms of the already-computed postfix string rather than scanning the infix expression a second time): each operand character becomes a new leaf node pushed onto the node stack; each operator pops its two most-recently-pushed subtrees as its left/right children, creates a new internal node for itself, and pushes that subtree back onto the stack. The single node left on the stack at the end is the root of the completed expression tree, which print_tree then displays with a recursive, indentation-based pre-order traversal.

Program 12 — Code

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#define STACK_IMPLEMENTATION
#include "stack.h"
#define BUFFER_LEN 64
typedef struct tree_node {
    char val;
    struct tree_node *left;
    struct tree_node *right;
} node_t;
void create_node(node_t *n, char val) {
    n->val = val;
    n->left = NULL;
    n->right = NULL;
}
void print_tree(node_t *n, int d) {
    if (!n) return;
    for (int i = 0; i < d; ++i)
        printf("    ");
    printf("%c\n", n->val);
    print_tree(n->left, d + 1);
    print_tree(n->right, d + 1);
}
void free_tree(node_t *n) {
    if (!n) return;
    free_tree(n->left);
    free_tree(n->right);
    free(n);
}
int precedence(char op) {
    switch (op) {
    case '+':
    case '-':
        return 1;
    case '*':
    case '/':
        return 2;
    case '^':
        return 3;
    default:
        return 0; // shouldn't be reached
    }
}
void itopfix(const char infix[BUFFER_LEN], char res[BUFFER_LEN]) {
    stack_t cstck; // stack of operator
    int i = 0;     // current string index
    stack_init(&cstck, sizeof(char), NULL);
    for (int j = 0; infix[j] != '\0'; ++j) {
        char c = infix[j];
        if (isspace(c))
            continue;
        if (isalnum(c))
            res[i++] = c;
        else if (c == '(')
            stack_push(&cstck, &c);

        else if (c == ')') {
            char d;
            stack_peek(&cstck, &d);
            while (d != '(') {
                stack_pop(&cstck, &d);
                res[i++] = d;
                stack_peek(&cstck, &d);
            }
            stack_pop(&cstck, &d); // pop '('
        }
        // Assume invalid characters will not be entered
        // Assume no variables
        else {
            char d;
            if (!stack_isempty(&cstck)) {
                stack_peek(&cstck, &d);
                while (precedence(d) >= precedence(c)) {
                    stack_pop(&cstck, &d);
                    res[i++] = d;
                    if (stack_isempty(&cstck))
```

```
                    break;
                stack_peek(&cstck, &d);
            }
        }
        stack_push(&cstck, &c);
    }
}
while (!stack_isempty(&cstck)) {
    char d;
    stack_pop(&cstck, &d);
    res[i++] = d;
}
res[i] = '\0'; // terminate the string
stack_free(&cstck);
}
node_t *peval(char postfix[BUFFER_LEN]) {
    stack_t dstck;
    stack_init(&dstck, sizeof(node_t *), NULL);

    for (int i = 0; postfix[i] != '\0'; ++i) {
        char c = postfix[i];
        if (isalnum(c)) {
            node_t *a = malloc(sizeof(node_t));
            create_node(a, c);
            stack_push(&dstck, &a);
        }
        // Operator otherwise
        else {
            // Only binary operators supported
            node_t *a, *b, *d;
            d = malloc(sizeof(node_t));
            stack_pop(&dstck, &b);
            stack_pop(&dstck, &a);
            create_node(d, c);
            d->left = a;
            d->right = b;
            stack_push(&dstck, &d);
        }
    }
    node_t *r;
    stack_pop(&dstck, &r);
    stack_free(&dstck);
    return r;
}

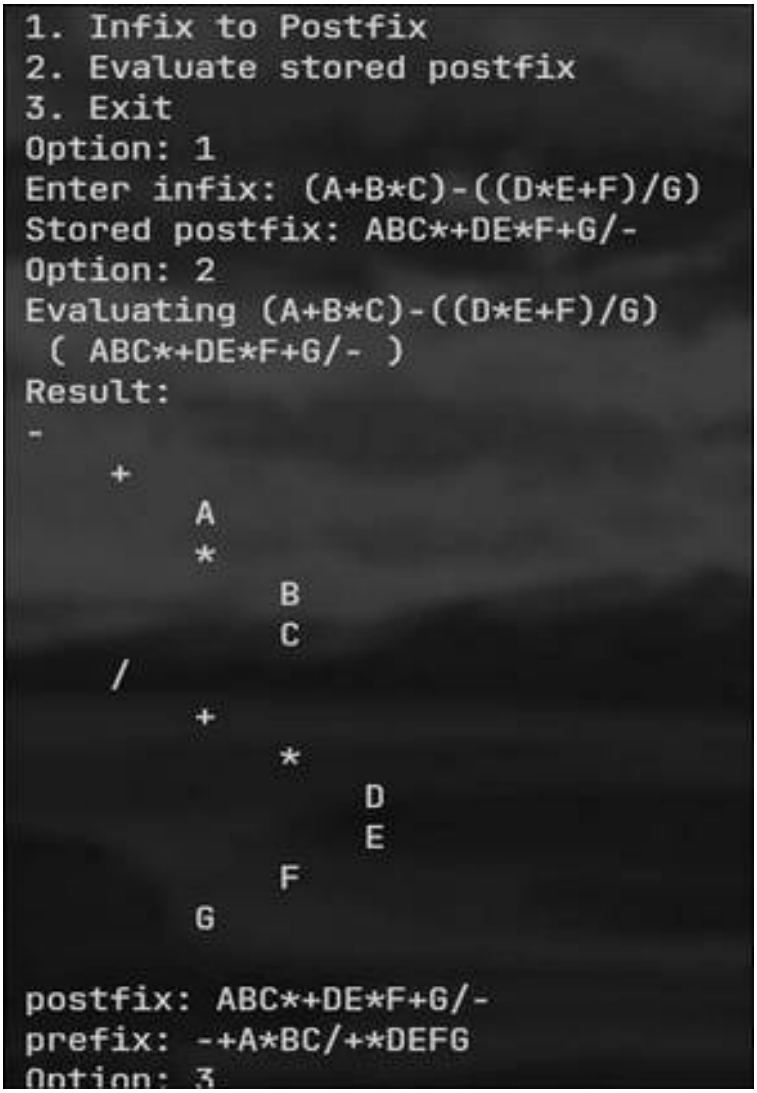
void postfix_print(node_t *root) {
       if (root == NULL) return;
       postfix_print(root→left);
       postfix_print(root→right);
       printf("%c", root→val);
}

void prefix_print(node_t *root) {
if (root == NULL) return;
printf("%c", root->val);
prefix_print(root->left);
prefix_print(root->right);
}

int main() {
    char infix[BUFFER_LEN] = {0};
    char postfix[BUFFER_LEN] = {0};
    int c, r;
    r = 1;
    printf("1. Infix to Postfix\n"
           "2. Evaluate stored postfix\n"
           "3. Exit\n");
    do {
        printf("Option: ");
        scanf("%d", &c);
        getc(stdin);
        switch (c) {
        case 1:
            printf("Enter infix: ");
            fgets(infix, BUFFER_LEN - 1, stdin);
            itopfix(infix, postfix);
            printf("Stored postfix: %s\n", postfix);
            break;
        case 2:
            printf("Evaluating %s ( %s )\n", infix, postfix);
```

```
            printf("Result: \n");
            node_t *res = peval(postfix);
            print_tree(res, 0);
            printf("\npostfix: ");
            postfix_print(res);
            printf("\nprefix: ");
            prefix_print(res);
            printf("\n");
            free_tree(res);
            break;
        case 3:
            r = 0;
            break;
        default:
            printf("Invalid option\n");
            break;
        }
    } while (r);
    return 0;
}
```

Program 12 — Output



Program 13 — Word–Meaning Dictionary using a BST

Question

Construct a binary tree to efficiently implement a dictionary of ⟨**`word, meaning`**⟩ *pairs. Have a menu to:*
(a) find the meaning of a given word
(b) insert a new ⟨**`word, meaning`**⟩ *pair*
(c) remove an existing ⟨**`word, meaning`**⟩ *pair.*

Data Structures / Algorithm Used

- **Data structure:** a self-balancing Binary Search Tree, keyed by the dictionary word via the comparator ***`cmp_entry`*** (***`strcmp`*** on word), storing ***`dict_entry_t { word, meaning }`*** pairs (heap-allocated strings, released through the ***`free_ele`*** callback ***`free_entry`*** on deletion).
- **Algorithms used:** the three menu operations map directly onto standard BST operations — insert (***`O(log n)`*** thanks to AVL rebalancing), search (an ***`O(log n)`*** binary search down the tree that copies out the matching entry or reports "Not found"), and delete.

Program 13 — Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define BST_IMPLEMENTATION
#include "bst.h"

#define W_LEN 32
#define M_LEN 256

struct word_meaning_pair {
    char *word;
    char *meaning;
};

typedef struct word_meaning_pair dict_entry_t;
typedef bst_t dict_t;

void create_entry(dict_entry_t *e, const char *word, const char *meaning) {
    e->word = strdup(word);
    e->meaning = strdup(meaning);
}

void free_entry(void *a) {
    dict_entry_t *e = (dict_entry_t *)a;
    free(e->word);
    free(e->meaning);
}

int cmp_entry(const void *a, const void *b) {
    const dict_entry_t *e1, *e2;
    e1 = (const dict_entry_t *)a;
    e2 = (const dict_entry_t *)b;
    return strcmp(e1->word, e2->word);
}

void dict_init(dict_t *d) {
    bst_init(d, sizeof(dict_entry_t), cmp_entry, free_entry);
}

const char *get_meaning(const dict_t *d, char *word) {
    dict_entry_t e;
    dict_entry_t k = {.word = word};
    int s = bst_get(d, &k, &e);
    return s ? e.meaning : "Not found";
}

void insert_meaning(dict_t *d, const char *word, const char *meaning) {
    dict_entry_t e;
    create_entry(&e, word, meaning);
    bst_insert(d, &e);
}

void remove_meaning(dict_t *d, char *word) {
    dict_entry_t k;
    k.word = word;
    bst_delete(d, &k);
}

int main() {
    dict_t dict;
    dict_init(&dict);

    printf("1. Insert meaning\n"
           "2. Get meaning\n"
           "3. Remove meaning\n"
           "4. Exit\n\n");
    int r = 1;
    int c;
    char w[W_LEN], m[M_LEN];
    do {
        scanf("%d", &c);
        getc(stdin);

        switch (c) {
        case 1:
            printf("Word: ");
            fgets(w, W_LEN, stdin);
```

```
            w[strcspn(w, "\n")] = '\0';

            printf("Meaning: ");
            fgets(m, M_LEN, stdin);
            m[strcspn(m, "\n")] = '\0';

            insert_meaning(&dict, w, m);
            break;
        case 2:
            printf("Word: ");
            fgets(w, W_LEN, stdin);
            w[strcspn(w, "\n")] = '\0';

            printf("Meaning: %s\n", get_meaning(&dict, w));
            break;
        case 3:
            printf("Word: ");
            fgets(w, W_LEN, stdin);
            w[strcspn(w, "\n")] = '\0';

            remove_meaning(&dict, w);
            break;
        case 4:
            r = 0;
            break;
        default:
            printf("Invalid option\n");
            break;
        }
    } while (r);
    bst_free(&dict);
    return 0;
}
```

Program 13 — Output

```
vijay@cet:~/Desktop/ds_lab$ ./a.out
1. Insert meaning
2. Get meaning
3. Remove meaning
4. Exit

1
Word: pomelics
Meaning: debate

2
Word: pomelics
Meaning: debate
2
Word: hi
Meaning: Not found
1
Word: hi
Meaning: A greeting
2
Word: hi
Meaning: A greeting
3
Word: hi
2
Word: hi
Meaning: Not found
4
```

Program 14 — Post Office Priority Queue

Question

The General post office wishes to give preferential treatment to its customers. The Postmaster has identified four categories of customers viz. Defence personnel, Differently abled persons, Senior citizens, and normal persons. Customers come to the post office, get their job done, and leave. As a customer enters the post office, he proceeds to a token machine which prompts for his category and then issues him a token bearing a number (in the increasing order of the time at which the customer arrived). The customers are to be given preference in the decreasing order - Differently abled persons, Senior citizens, Defence personnel, and Normal persons. Customers within the same category are to be served in the order of arrival. Implement this scenario for N customers. You may assume that there is only one service counter in the post office.

Data Structures / Algorithm Used

- **Data structure:** a max-heap of ***`token_t { token_id, priority, name }`***.
- **Algorithm used:** the heap's ordering is driven entirely by a custom comparator token_cmp that first compares category priority (Differently-abled > Senior citizen > Defence personnel > Normal, per the question's required preference order) and, for two tokens of equal category, compares token ids so that the earlier-arrived customer (smaller token_id) is treated as "greater" — this makes popmax always extract the highest-priority, earliest-arrived customer, correctly implementing multi-level (category, then FIFO-within-category) priority scheduling with a single binary heap.

Program 14 — Code

```
#include <stdio.h>
#include <string.h>
#define HEAP_IMPLEMENTATION
#include "heap.h"
#define LEN 64
enum Priority { NORMAL, DEFENCE, SENIOR, DIFFERENTLY_ABLED };
const char *ptos[4] = {"NORMAL", "DEFENCE", "SENIOR", "DIFFERENTLY ABLED"};
struct post_office_token {
    int token_id;
    enum Priority p;
    char name[LEN];
};
typedef struct post_office_token token_t;
int token_cmp(const void *a, const void *b) {
    const token_t *t1, *t2;
    t1 = (const token_t *)a;
    t2 = (const token_t *)b;
    int d = t1->p - t2->p;
    return d ? d : t2->token_id - t1->token_id;
}
void print_token(token_t t) {
    printf("%d. %s: %s\n", t.token_id, t.name, ptos[t.p]);
}
int main() {
    hp_t pq;
    hp_init(&pq, sizeof(token_t), token_cmp, NULL);

    printf("Normal: 0\n"
           "Defense: 1\n"
           "Senior: 2\n"
           "Differently abled: 3\n\n");

    printf("1. Take token\n"
           "2. Serve customer\n"
           "3. Exit\n\n");
    int c, r, t, s;
    token_t tk;
    t = 1;
    r = 1;
    do {
        scanf("%d", &c);
        getc(stdin);
        switch (c) {
        case 1:
            printf("Enter name: ");
            fgets(tk.name, LEN, stdin);
            tk.name[strcspn(tk.name, "\n")] = '\0';

            printf("Enter priority: ");
            scanf("%d", (int *)&tk.p);
            getc(stdin);

            tk.token_id = t++;
            hp_insert(&pq, &tk);
            printf("Inserted: ");
            print_token(tk);
            break;
        case 2:
            s = hp_popmax(&pq, &tk);
            printf("Served: ");
            if (s)
                print_token(tk);
            else
                printf("Empty\n");
            break;
        case 3:
            r = 0;
            break;
        default:
            printf("Invalid choice\n");
            break;
        }

    } while (r);
    hp_free(&pq);
    return 0;
}
```

Program 14 — Output

```
vijay@cet:~/Desktop/ds_lab$ ./a.out
Normal: 0
Defense: 1
Senior: 2
Differently abled: 3

1. Take token
2. Serve customer
3. Exit

1
Enter name: Vijay
Enter priority: 0
Inserted: 1. Vijay: NORMAL
1
Enter name: Vaishak
Enter priority: 2
Inserted: 2. Vaishak: SENIOR
1
Enter name: Steve
Enter priority: 0
Inserted: 3. Steve: NORMAL
2
Served: 2. Vaishak: SENIOR
2
Served: 1. Vijay: NORMAL
2
Served: 3. Steve: NORMAL
3
```

Program 15 — Merge K Sorted Lists using a Min-Heap

Question

Merge K sorted lists into a single sorted list using a heap. Use a min-heap to keep track of the smallest element from each list. Repeatedly extract the smallest element and insert the next element from the corresponding list into the heap until all lists are merged.

Data Structures / Algorithm Used

- **Data structure:** a binary heap of ***`arr_view { arr, i, n }`*** structs, one per input list, each tracking a pointer to that list's (pre-sorted) array along with a "current position" cursor ***`i`***. The heap's comparator ***`min_cmp`*** is deliberately inverted, so that the library's max-heap (***`popmax`***) behaves as the min-heap the algorithm needs.
- **Algorithm used** — K-way merge with a heap: every one of the k input arrays is first sorted independently (quick sort), and an ***`arr_view`*** positioned at its smallest element is pushed into the heap. The loop then repeatedly: pops the view whose current element is the smallest among all k lists, appends that element to the output, advances that view's cursor by one, and (if the list isn't exhausted) re-inserts the view into the heap — this is exactly the textbook ***`O((n·k)·log k)`*** K-way merge using a min-heap, where ***`n·k`*** is the total number of elements across all lists.

Program 15 — Code

```
#include <alloca.h> // alloca
#include <stddef.h> // size_t
#include <stdio.h>
#define HEAP_IMPLEMENTATION
#include "heap.h"
#define QUICK_SORT_IMPL
#include "sort.h"

struct arr_view {
    int *arr; // array start
    size_t i; // current view
    size_t n; // capacity
};

int min_cmp(const void *a, const void *b) {
    const struct arr_view *c, *d;
    c = (const struct arr_view *)a;
    d = (const struct arr_view *)b;
    return d->arr[d->i] - c->arr[c->i];
}

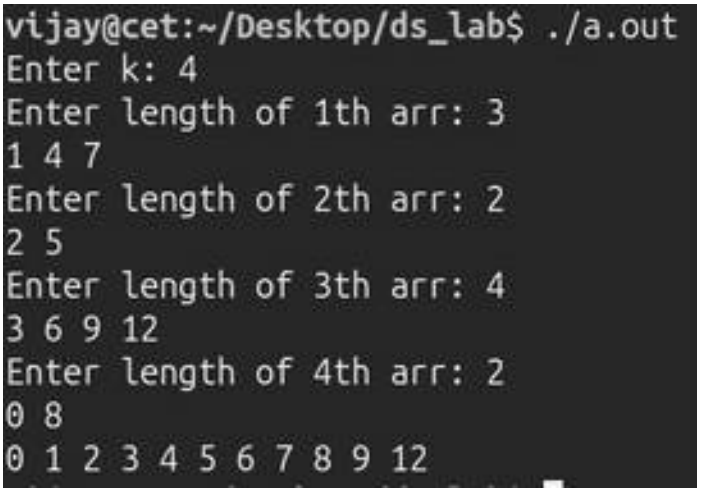
int int_cmp(const void *a, const void *b) {
    return *(const int *)a - *(const int *)b;
}

int main() {
    int k, c, i;
    struct arr_view av;
    hp_t heap;
    hp_init(&heap, sizeof(struct arr_view), min_cmp, NULL);
    printf("Enter k: ");
    scanf("%d", &k);
    int *arrs[k];
    for (i = 0; i < k; ++i) {
        int n;
        printf("Enter length of %dth arr: ", i + 1);
        scanf("%d", &n);
        arrs[i] = alloca(sizeof(int) * n);
        for (int j = 0; j < n; ++j)
            scanf("%d", &arrs[i][j]);
        quick_sort(arrs[i], n, sizeof(int), int_cmp);
        av.arr = arrs[i];
        av.i = 0;
        av.n = n;
        hp_insert(&heap, &av);
        c += n;
    }

    int res[c];
    i = 0;
    while (hp_popmax(&heap, &av)) {
        if (av.i == av.n)
            continue;
        res[i++] = av.arr[av.i++];
        hp_insert(&heap, &av);
    }

    for (i = 0; i < c; ++i)
        printf("%d ", res[i]);
    printf("\n");
    hp_free(&heap);
    return 0;
}
```

Program 15 — Output



Program 16 — Shortest Path in a Maze (A* Search)

Question

Assume our department owns a robot DotSlash and you arrange a "Robo-show" to showcase its maze solving capability!! At one instant of the show, the robo is currently at a point as shown in Figure 3. The poor guy is running out of charge and somehow wants to reach the charging source (shown in the figure). It now seeks your help for the same. Since the robo has got only a few units of charge left in its battery, you want to make sure that the robo consumes the least amount of charge in its journey to the power source. ***Assume that the charge consumed is directly proportional to the distance traversed. Help DotSlash to find a path to the charging point, minimizing the battery power consumed.***

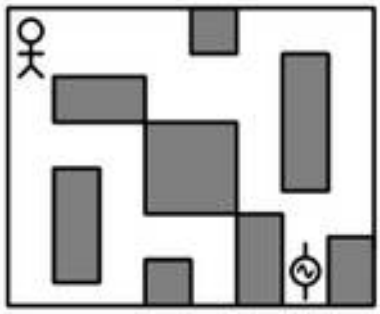


Figure 3

Data Structures / Algorithm Used

- **Data structure:** a binary max-heap used as a priority queue ("open set") ordered by ascending ***f-score*** via an inverted comparator, plus 2-D arrays for ***gScore, fScore, and cameFrom***, all sized to the maze's dimensions; the reconstructed path is collected into a dynamic array.
- **Algorithm used** — ***A**** path finding algorithm: Starting from the robot's position, it repeatedly extracts the open-set cell with the lowest ***f-score*** (= cost so far, ***g-score*** plus a ***Manhattan-distance*** heuristic to the goal) and relaxes each of its 4 grid neighbours (skipping obstacles and out-of-bounds cells): if reaching a neighbour through the current cell is cheaper than any route found so far, its ***g-score, f-score*** and ***came-from*** pointer are updated and it is pushed into the open set. The search stops as soon as the goal cell is popped, and the shortest path is reconstructed by following the ***came-from*** pointers back to the start and reversing the result.

Program 16 — Code

```
#include <stdio.h>
#include <stdlib.h> // abs
#include <limits.h>
#define HEAP_IMPLEMENTATION
#include "heap.h"
#define LIST_IMPLEMENTATION
#include "list.h"

#define MAP_VAL(map, pt) ((map)[(pt).x][(pt).y])

typedef struct {
    int x;
    int y;
} point_t;

struct heap_struct {
    point_t p;
    int score;
};

int heap_cmp(const void *a, const void *b) {
    return ((const struct heap_struct *)b)->score - ((const struct heap_struct *)a)->score;
}

int dist(const point_t src, const point_t dst) {
    return abs(src.x - dst.x) + abs(src.y - dst.y);
}

int cmp(const point_t a, const point_t b) {
    return dist(a, b) == 0;
}

void rev(point_t *p, int n) {
    for (int i = 0; i < n / 2; ++i) {
        point_t t = p[i];
        p[i] = p[n - 1 - i];
        p[n - 1 - i] = t;
    }
}

int is_valid(int m, int n, int grid[m][n], point_t p) {
    if (p.x < 0 || p.x >= m) return 0;
    if (p.y < 0 || p.y >= n) return 0;
    return grid[p.x][p.y] != 1;
}

list_t a_star(int m, int n, int grid[m][n], point_t start, point_t goal) {
    hp_t open;
    hp_init(&open, sizeof(struct heap_struct), heap_cmp, NULL);

    point_t (*cameFrom)[n] = malloc(sizeof(point_t) * m * n);
    int (*gscore)[n] = malloc(sizeof(int) * m * n);
    int (*fscore)[n] = malloc(sizeof(int) * m * n);

    for(int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            cameFrom[i][j] = (point_t) {.x = -1, .y = -1};
            gscore[i][j] = INT_MAX;
            fscore[i][j] = INT_MAX;
        }
    }

    MAP_VAL(gscore, start) = 0;
    MAP_VAL(fscore, start) = dist(start, goal);

    struct heap_struct cur = (struct heap_struct) {.p = start, .score = MAP_VAL(fscore, start)};
    hp_insert(&open, &cur);

    list_t path;
    list_init(&path, 3, sizeof(point_t), NULL);

    while (hp_peek(&open, NULL)) {
        hp_popmax(&open, &cur);
        point_t cur_p = cur.p;

        if (cmp(cur_p, goal))
            goto path_reconstruct;
```

```
        point_t neighbours[4];

        neighbours[0].x = cur_p.x + -1;
        neighbours[0].y = cur_p.y + 0;

        neighbours[1].x = cur_p.x + 0;
        neighbours[1].y = cur_p.y + -1;

        neighbours[2].x = cur_p.x + 0;
        neighbours[2].y = cur_p.y + 1;

        neighbours[3].x = cur_p.x + 1;
        neighbours[3].y = cur_p.y + 0;

        for (int i = 0; i < 4; ++i) {
            point_t neighbour = neighbours[i];
            if (!is_valid(m, n, grid, neighbour))
                continue;
            int tentative_gscore = MAP_VAL(gscore, cur_p) + 1;

            if (tentative_gscore < MAP_VAL(gscore, neighbour)) {
                MAP_VAL(cameFrom, neighbour) = cur_p;
                MAP_VAL(gscore, neighbour) = tentative_gscore;
                MAP_VAL(fscore, neighbour) = tentative_gscore + dist(neighbour, goal);

                struct heap_struct nxt = { .p = neighbour, .score = MAP_VAL(fscore, neighbour) };

                hp_insert(&open, &nxt);
            }

        }
    }
    goto free_ret;

    point_t cur_p;
path_reconstruct:
    cur_p = cur.p;
    add(&path, &cur_p);
    while (MAP_VAL(cameFrom, cur_p).x != -1) {
        cur_p = MAP_VAL(cameFrom, cur_p);
        add(&path, &cur_p);
    }
    rev((point_t *)path.data, path.size);
free_ret:
    free(gscore);
    free(fscore);
    free(cameFrom);
    hp_free(&open);
    return path;
}

int main() {
    int maze[8][8] = {
        {0, 0, 0, 0, 1, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 1, 0},
        {0, 1, 1, 0, 0, 0, 1, 0},
        {0, 0, 0, 1, 1, 0, 1, 0},
        {0, 1, 0, 1, 1, 0, 0, 0},
        {0, 1, 0, 0, 0, 1, 0, 0},
        {0, 1, 0, 1, 0, 1, 0, 1},
        {0, 1, 0, 1, 0, 1, 0, 1}
    };
    point_t start = (point_t) {.x = 0, .y = 0};
    point_t goal = (point_t) {.x = 7, .y = 6};
    list_t result = a_star(8, 8, maze, start, goal);

    for (int i = 0; i < list_size(&result); ++i) {
        point_t c;
        get(&result, i, &c);
        printf(" (%d, %d) -> ", c.y, c.x);  // x and y got inverted;
    }
    printf(list_size(&result) ? "!\n" : "X\n");

    return 0;
}
```

Program 16 — Output

```
vijay@cet:~/Desktop/ds_lab$ head -n 138 pgm/pgm_16.c | tail -n 10
    int maze[8][8] = {
        {0, 0, 0, 0, 1, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 1, 0},
        {0, 1, 1, 0, 0, 0, 1, 0},
        {0, 0, 0, 1, 1, 0, 1, 0},
        {0, 1, 0, 1, 1, 0, 0, 0},
        {0, 1, 0, 0, 0, 1, 0, 0},
        {0, 1, 0, 1, 0, 1, 0, 1},
        {0, 1, 0, 1, 0, 1, 0, 1}
    };
vijay@cet:~/Desktop/ds_lab$ ./a.out
 (0, 0) ->  (1, 0) ->  (1, 1) ->  (2, 1) ->  (3, 1) ->  (3, 2) ->  (4, 2) ->  (5, 2) ->  (5, 3) ->  (5, 4) ->  (6
, 4) ->  (6, 5) ->  (6, 6) ->  (6, 7) -> !
```

Program 17 — The Water Jug Problem (BFS State-Space Search)

Question

The water jug problem is a classic puzzle in the domain of Artificial Intelligence. It involves determining how to measure a specific quantity of water using two or more jugs with different capacities. The challenging part is that none of the jugs have any volume markings on them. The goal here is to find a sequence of actions (filling, emptying, or pouring between the jugs) that leads to the desired measurement. Now let me make it formal:
You are given three containers whose sizes are 10 litres, 7 litres, and 4 litres, respectively. The 7-litre and 4-litre containers start out full of water, but the 10-litre container is initially empty. You are allowed to perform only one type of operation: pouring the contents of one container into another, stopping when the source container is empty or the destination container is full. Your task is to determine if there is a sequence of pourings that leaves exactly 2 litres in the 7-litre or 4-litre container.

Data Structures / Algorithm Used

- **Data structure:** a queue holding ***`overall_state { js, prev }`*** nodes for BFS, plus a hash-based set of already-visited ***`jug_state`*** triples to prevent re-exploring the same configuration.
- **Algorithm used** — Breadth-First Search over the state space of the three jugs (10L, 7L, 4L): starting from the initial state, the six possible pour operations (each jug into each of the other two) are used to generate successor states, which are enqueued together with a prev pointer back to their parent state (for path reconstruction). States are dequeued in FIFO order (guaranteeing BFS explores states in increasing number of pours, i.e. finds the shortest sequence), and the visited set ensures each distinct (***`c10, c7, c4`***) configuration is expanded at most once. As soon as a dequeued state satisfies the target condition (***`c7 == 2 or c4 == 2`***), the prev chain is walked backwards and reversed into a list to reconstruct and print the actual sequence of pourings from the start state to the target.

Program 17 — Code

```
#include <stdio.h>
#define LIST_IMPLEMENTATION
#include "list.h"
#define QUEUE_IMPLEMENTATION
#include "queue.h"
#define SET_IMPLEMENTATION
#include "set.h"

int min(int a, int b) { return a < b ? a : b; }
struct jug_state {
    int c10;
    int c7;
    int c4;
};
struct overall_state {
    struct jug_state js;
    struct overall_state *prev;
};
typedef struct jug_state state_t;

void rev(state_t *a, int n) {
    for (int i = 0; i < n / 2; ++i) {
        state_t t = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = t;
    }
}

list_t search_space(state_t init, int (*istarget)(const state_t)) {
    queue_t bfs;
    set_t visited;

    q_init(&bfs, sizeof(struct overall_state), NULL);
    set_init(&visited, sizeof(state_t), NULL);

    struct overall_state cur_state = {.js = init, .prev = NULL};
    enqueue(&bfs, &cur_state);
    while (!q_isempty(&bfs)) {
        state_t cur;
        dequeue(&bfs, &cur_state);
        cur = cur_state.js;
        if (set_contains(&visited, &cur))
            continue;
        if (istarget(cur))
            goto path_construct;
        set_insert(&visited, &cur);

        struct overall_state nxt_states[12];
        int amt;
        for (int i = 0; i < 6; ++i) {
            nxt_states[i].prev = malloc(sizeof(struct overall_state));
            *nxt_states[i].prev = cur_state;
            nxt_states[i].js = cur;
        }
        // Pour c10 to c7
        amt = min(cur.c10, 7 - cur.c7);
        nxt_states[0].js.c10 -= amt;
        nxt_states[0].js.c7 += amt;

        // Pour c10 to c4
        amt = min(cur.c10, 4 - cur.c4);
        nxt_states[1].js.c10 -= amt;
        nxt_states[1].js.c4 += amt;

        // Pour c7 to c10
        amt = min(cur.c7, 10 - cur.c10);
        nxt_states[2].js.c7 -= amt;
        nxt_states[2].js.c10 += amt;

        // Pour c7 to c4
        amt = min(cur.c7, 4 - cur.c4);
        nxt_states[3].js.c7 -= amt;
        nxt_states[3].js.c4 += amt;

        // Pour c4 to c10
        amt = min(cur.c4, 10 - cur.c10);
        nxt_states[4].js.c4 -= amt;
```

```
        nxt_states[4].js.c10 += amt;

        // Pour c4 to c7
        amt = min(cur.c4, 7 - cur.c7);
        nxt_states[5].js.c4 -= amt;
        nxt_states[5].js.c7 += amt;

        for (int i = 0; i < 6; ++i)
            enqueue(&bfs, &nxt_states[i]);
    }
    goto free_ret;
    list_t path;
path_construct:
    list_init(&path, 3, sizeof(state_t), NULL);
    do {
        add(&path, &cur_state.js);
        cur_state = *cur_state.prev;
    } while (cur_state.prev != NULL);
    add(&path, &cur_state.js);
    rev((state_t *)path.data, path.size);
free_ret:
    q_free(&bfs);
    set_free(&visited);
    return path;
}

int istarget(const state_t s) {
#ifdef ORIGINAL
    return s.c7 == 2 || s.c4 == 2;
#endif
#ifdef DBG1
    return s.c10 == 2 && s.c7 == 5 && s.c4 == 3; // unreachable
#endif
#ifdef DBG2
    return s.c10 == 8 && s.c7 == 3 && s.c4 == 0; // reachable
#endif
    return 0;
}

int main() {
    list_t result;
    state_t init = (state_t){.c10 = 0, .c7 = 7, .c4 = 4};
    result = search_space(init, istarget);

    printf(
#ifdef ORIGINAL
            "target = (X 2 X) or (X X 2)\n"
#endif
#ifdef DBG1
            "target = (2 5 3) (unreachable)\n"
#endif
#ifdef DBG2
            "target = (8 3 0) (reachable)\n"
#endif
            ""
          );

    for (int i = 0; i < list_size(&result); ++i) {
        state_t cur;
        get(&result, i, &cur);
        printf(" (%d %d %d) -> ", cur.c10, cur.c7, cur.c4);
    }
    printf(list_size(&result) ? " !\n" : "X\n");
    return 0;
}
```

Program 17 — Output

```
vijay@cet:~/Desktop/ds_lab$ gcc -DORIGINAL -I headers/ pgm/pgm_17.c
vijay@cet:~/Desktop/ds_lab$ ./a.out
target = (X 2 X) or (X X 2)
 (0 7 4) ->  (4 7 0) ->  (10 1 0) ->  (6 1 4) ->  (6 5 0) ->  (2 5 4) ->  (2 7 2) ->  !
vijay@cet:~/Desktop/ds_lab$ gcc -DDBG1 -I headers/ pgm/pgm_17.c
vijay@cet:~/Desktop/ds_lab$ ./a.out
target = (2 5 3) (unreachable)
X
vijay@cet:~/Desktop/ds_lab$ gcc -DDBG2 -I headers/ pgm/pgm_17.c
vijay@cet:~/Desktop/ds_lab$ ./a.out
target = (8 3 0) (reachable)
 (0 7 4) ->  (4 7 0) ->  (4 3 4) ->  (8 3 0) ->  !
```

Program 18 — Sorting Algorithms and Step-Count Comparison

Question

Write a menu-driven program to read numerical data stored in a file and implement the following algorithms for sorting the numbers in descending order. Implement each algorithm as a separate function.
(a) Bubble sort
(b) Insertion Sort
(c) Radix Sort
(d) Merge Sort
(e) Quick Sort
Compare the number of steps involved in each method.

Data Structures / Algorithm Used

- **Data structure:** plain in-memory int arrays (numbers are read from a file given on the command line).
- **Algorithms used** — five classical comparison/non-comparison sorts, each producing descending order, instrumented with a step counter (***`cmp/shift/swp`*** helpers increment a shared counter whenever ***`COUNT_CMP/COUNT_SHF`*** is enabled at compile time):
- **Bubble Sort** — repeated adjacent-pair comparisons/swaps, with early termination once a full pass makes no swap.
- **Insertion Sort** — shifts larger elements one position to the right to make room for the current key.
- **Radix Sort** — an LSD (least-significant-digit) radix sort built from a stable counting sort per decimal digit, repeated once per digit place of the largest number.
- **Merge Sort** — classic divide-and-conquer, using temporary buffers in the merge step.
- **Quick Sort** — partition scheme (pivot = last element), recursing on both partitions.

Each algorithm's total instrumented step count is printed so the five methods can be empirically compared on the same input.

Program 18 — Code

```
#include <alloca.h>
#include <string.h>
#include <stdio.h>
#include <assert.h>

#ifdef COUNT_TOT
#define COUNT_CMP
#define COUNT_SHF
#endif

#if !defined(COUNT_CMP) && !defined(COUNT_SHF)
#error "You are not counting anything! Define COUNT_CMP, COUNT_SHF or COUNT_TOT..."
#endif

// define COUNT_CMP to count comparisons
// define COUNT_SHF to count shifts

// Due to the ambiguousity of the question on the
// definition of a 'step', you may notice double counting or
// even triple counting based on the particular definition
// adopted. But the overall growth should still be reflected
// as the differences are only linear

int cmp(int a, int b, int *s) {
#ifdef COUNT_CMP
    (*s)++;
#endif
    return a - b;
}

void swp(int *a, int *b, int *s) {
#ifdef COUNT_SHF
    (*s) += 3;
#endif
    int t = *a;
    *a = *b;
    *b = t;
}

void shift(int *a, int b, int *s) {
#ifdef COUNT_SHF
    (*s)++;
#endif
    *a = b;
}

int bubble_sort(int *a, int n) {
    int c = 0;

    for (int i = 0; i < n; ++i) {
        int s = 0;

        for (int j = 0; j < n - i - 1; ++j) {
            if (cmp(a[j], a[j + 1], &c) < 0) {
                swp(&a[j], &a[j + 1], &c);
                s = 1;
            }
        }
        if (!s)
            break;
    }

    return c;
}

int insertion_sort(int *a, int n) {
    if (n < 2)
        return 0;
    int c = 0;

    for (int i = 1; i < n; ++i) {
        int k = a[i];
        int j;
        for (j = i - 1; j >= 0 && cmp(a[j], k, &c) < 0; --j)
            shift(&a[j + 1], a[j], &c);
        shift(&a[j + 1], k, &c);
    }
```

```
    return c;
}

int count_sort_by_digit(int *a, int n, int place) {
    int c = 0;

    int out[n];
    int cnt[10] = {0};

    for (int i = 0; i < n; ++i) {
        int d;
        shift(&d, 9 - (a[i] / place) % 10, &c);
        cnt[d]++;
    }
    for (int i = 1; i < 10; ++i)
        cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; --i) {
        int d;
        shift(&d, 9 - (a[i] / place) % 10, &c);
        shift(&out[cnt[d] - 1], a[i], &c);
        cnt[d]--;
    }
    for (int i = 0; i < n; ++i)
        shift(&a[i], out[i], &c);
    return c;
}

int radix_sort(int *a, int n) {
    if (n == 0)
        return 0;
    int c = 0;
    int max = a[0];
    for (int i = 1; i < n; ++i)
        max = cmp(max, a[i], &c) > 0 ? max : a[i];
    int place = 1;
    while (max / place) {
        c += count_sort_by_digit(a, n, place);
        place *= 10;
    }
    return c;
}

int merge(int *a, int l, int m, int r) {
    int c = 0;
    int o, p;
    o = m - l + 1;
    p = r - m;
    int *lp = alloca(sizeof(int) * o);
    int *rp = alloca(sizeof(int) * p);
    for (int i = 0; i < o; ++i)
        shift(&lp[i], a[l + i], &c);
    for (int i = 0; i < p; ++i)
        shift(&rp[i], a[m + 1 + i], &c);
    int i, j, k;
    i = j = k = 0;
    while (i < o && j < p) {
        if (cmp(lp[i], rp[j], &c) > 0) {
            shift(&a[l + k], lp[i], &c);
            i++;
        } else {
            shift(&a[l + k], rp[j], &c);
            j++;
        }
        k++;
    }
    while (i < o) {
        shift(&a[l + k], lp[i], &c);
        i++;
        k++;
    }

    while (j < p) {
        shift(&a[l + k], rp[j], &c);
        j++;
        k++;
    }

    return c;
}
```

```
int merge_sort(int *a, int l, int r) {
    int c = 0;
    if (l >= r)
        return 0;

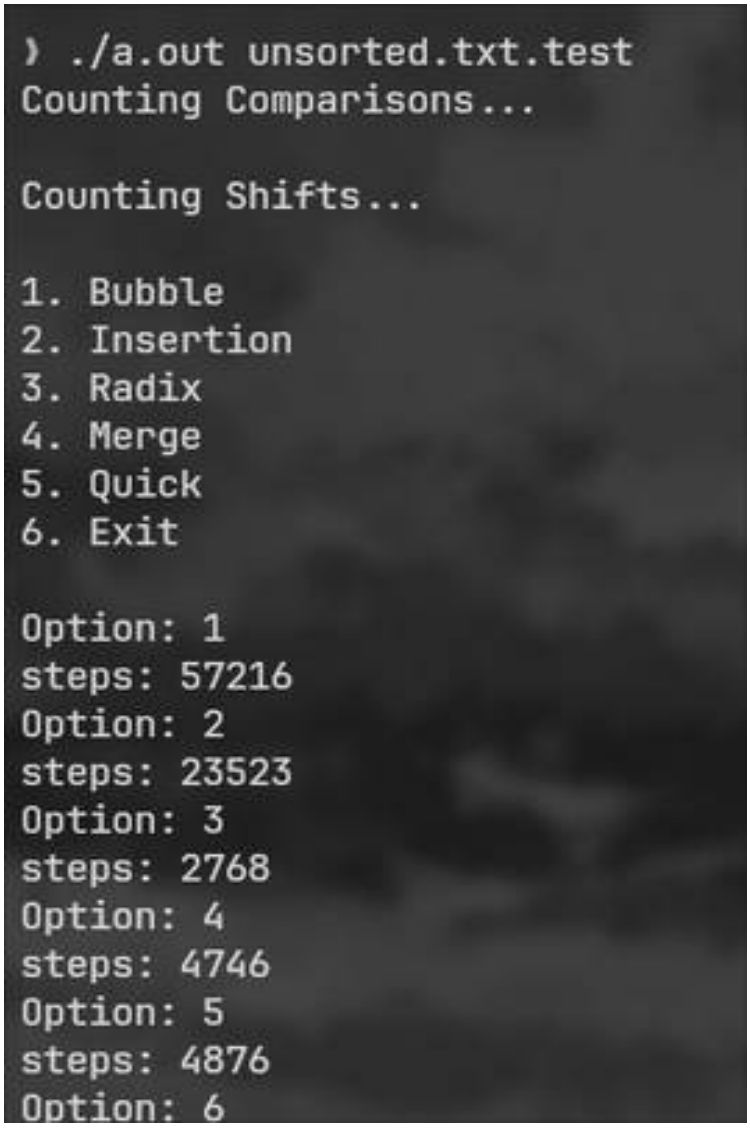
    int m = (l + r) / 2;
    c += merge_sort(a, l, m);
    c += merge_sort(a, m + 1, r);
    c += merge(a, l, m, r);
    return c;
}

int partition(int *a, int l, int h, int *c) {
    int p;
    shift(&p, a[h], c);
    int i = l - 1;
    for (int j = l; j < h; ++j) {
        if (cmp(a[j], p, c) > 0) {
            ++i;
            swp(&a[i], &a[j], c);
        }
    }
    swp(&a[i + 1], &a[h], c);
    return i + 1;
}

int quick_sort(int *a, int l, int h) {
    if (l >= h)
        return 0;
    int c = 0;
    int p = partition(a, l, h, &c);
    c += quick_sort(a, l, p - 1);
    c += quick_sort(a, p + 1, h);
    return c;
}

int main(int argc, char **argv) {
    assert(argc == 2 && "./a.out 'unsorted.txt'");
    int n = 0;
    FILE *fh = fopen(argv[1], "r");
    char line[8];
    while (fgets(line, 8, fh)) n++;
    int *a = alloca(sizeof(int) * n);
    int *b = alloca(sizeof(int) * n);
    rewind(fh);
    int i = 0;
    while (fgets(line, 8, fh)) {
        sscanf(line, "%d\n", &a[i++]);
    }
    fclose(fh);
    int s;
#ifdef COUNT_CMP
    printf("Counting Comparisons...\n");
#endif
#ifdef COUNT_SHF
    printf("Counting Shifts...\n");
#endif
       printf("1. Bubble\n2. Insertion\n3. Radix\n4. Merge\n5. Quick\n6. Exit\n\n");
       int o,r;
       r = 1;
       while (r) {
              printf("Option: ");
              scanf("%d", &o);
              memcpy(b, a, sizeof(int) * n);
              switch (o) {
                     case 1:s = bubble_sort(b, n);break;
                     case 2:s = insertion_sort(b, n);break;
                     case 3:s = radix_sort(b, n);break;
                     case 4:s = merge_sort(b, 0, n - 1);break;
                     case 5:s = quick_sort(b, 0, n - 1);break;
                     case 6:r = 0;break;
                     default:printf("Invalid option\n");break;
              }
              if (o < 6) printf("steps: %d\n", s);
       }
    return 0;
}
```

Program 18 — Output



Program 19 — Maximum Activity Points with At Most K Events

Question

Our department is organizing a tech fest with so many exciting events. By participating in an event, you can claim for activity points as stipulated by KTU. Each event `i` *gives you* `A[i]` *activity points where* `A` *is an array. If you are not allowed to participate in more than* `k` *events, what's the max number of points that you can earn?*

Data Structures / Algorithm Used

- **Data structure:** a small fixed-size auxiliary array `B[k]`, always kept sorted in descending order, representing the current best `k` activity-point values seen so far.
- **Algorithm used** — "keep the top-k" via insertion: for every incoming event value `a`, it is compared left-to-right against `B`; the first position `j` where `a > B[j]` is found, all elements from `j` onward are shifted one place to the right (discarding the smallest currently-kept value, `B[k−1]`), and `a` is written into `B[j]`. Since there is no ordering/adjacency restriction on which events may be chosen, the optimal strategy is simply to select the k highest-valued events — which is exactly what this insertion-based "maintain the top-k" array achieves in `O(n·k)` time — and the final answer is `sum(B)`.

Program 19 — Code

```
#include <stdio.h>

int main() {
    int n, k;
    printf("Enter max events participatable: ");
    scanf("%d", &k);
    printf("Enter number of events: ");
    scanf("%d", &n);
    int A[n + k];
    int *B = &A[n];

    for (int i = 0; i < n; ++i)
        scanf("%d", &A[i]);

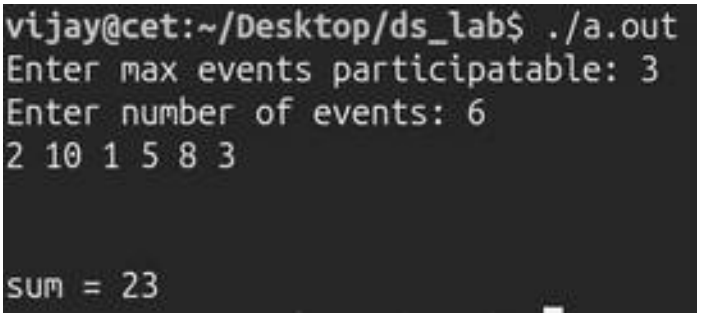
    for (int i = 0; i < k; ++i)
        B[i] = 0;

    for (int i = 0; i < n; ++i) {
        int a = A[i];
        for (int j = 0; j < k; ++j) {
            if (a > B[j]) {
                for (int m = k - 1; m > j; --m)
                    B[m] = B[m - 1];
                B[j] = a;
                break;
            }
        }
    }

    int sum = 0;
    for (int i = 0; i < k; ++i)
        sum += B[i];
    printf("\n\nsum = %d\n", sum);

    return 0;
}
```

Program 19 — Output



Program 20 — Spell Checker using a Hash Table

Question

Hash tables have a plethora of applications. Let us now explore one. You are given the task of implementing a spell checker. First, read a file ***`vocabulary.txt`*** *containing valid words. Use an appropriate hash function to index them into a hash table sized 29, employing chaining for collision resolution. Next, read a second file* ***`myDocument.txt`****, and determine the count of misspelt words in it. Also suggest corrections for the misspelt words. You may assume that* ***`myDocument.txt`*** *consists only of (possibly misspelt) valid words.*

Data Structures / Algorithm Used

- **Data structure:** a hash set with a custom string hash and comparator, storing the vocabulary as a set of unique words; a dynamic array separately holds every vocabulary word (needed to scan candidates for correction suggestions, since a hash set cannot be iterated in a useful order).
- **Algorithm used:** every word of ***`vocabulary.txt`*** is inserted into both the hash set (for ***`O(1)`*** average membership testing) and the list (for iteration). Every word of ***`myDocument.txt`*** is looked up in the hash set; any word not found is counted as a misspelling, and the ***`Levenshtein`*** edit distance is computed against every word in the vocabulary list — if the closest match is within an edit distance of 3, it is offered as a suggested correction.

Program 20 — Code

```
#include <assert.h>
#include <stdio.h>
#include <string.h>
#define SET_IMPLEMENTATION
#include "set.h"
#define LIST_IMPLEMENTATION
#include "list.h"
#define LEN 64

int min(size_t a, size_t b, size_t c) {
    return (a < b) ? ((a < c) ? a : c) : ((b < c) ? b : c);
}

// Levenshtein Distance
size_t edit_dist(const char *a, const char *b) {
    size_t m, n;
    m = strlen(a);
    n = strlen(b);
    size_t D[m + 1][n + 1];
    size_t i, j;
    for (j = 0; j <= n; ++j)
        D[0][j] = j;
    for (i = 0; i <= m; ++i)
        D[i][0] = i;
    size_t c;
    for (i = 1; i <= m; ++i) {
        for (j = 1; j <= n; ++j) {
            c = (a[i - 1] == b[j - 1]) ? 0 : 1;
            D[i][j] = min(
                    D[i - 1][j] + 1,    // delete
                    D[i][j - 1] + 1,    // insert
                    D[i - 1][j - 1] + c // substitute
                    );
        }
    }
    return D[m][n];
}

size_t string_hash(const void *a, size_t size) {
    size_t h = __hash(*(const char **)a, strlen(*(const char **)a));
    return h;
}

int string_cmp(const void *a, const void *b, size_t s) {
    const char *s1 = *(const char **)a;
    const char *s2 = *(const char **)b;
    int r = strcmp(s1, s2);
    return r;
}

void string_free(void *a) { free(*(char **)a); }

int main(int argc, char **argv) {
    assert(argc == 3 && "Usage: ./a.out 'vocabulary.txt' 'document.txt'");

    set_t dict;
    list_t all_words;
    set_init(&dict, sizeof(char *), string_free);
    set_hash(&dict, string_hash);
    set_cmp(&dict, string_cmp);
    list_init(&all_words, 3, sizeof(char *), NULL); // set will free
    FILE *fh;
    char line[LEN];
    // fill dictionary
    fh = fopen(argv[1], "r");
    assert(fh);

    while (fgets(line, LEN, fh)) {
        line[strcspn(line, "\n")] = '\0';
        char *s = strdup(line);
        set_insert(&dict, &s);
        add(&all_words, &s);
    }

    fclose(fh);
    // now read and count mistakes
    int c = 0;
```

```
    fh = fopen(argv[2], "r");
    assert(fh);

    while (fgets(line, LEN, fh)) {
        line[strcspn(line, "\n")] = '\0';
        char *s = strdup(line);
        if (!set_contains(&dict, &s)) {
            char *min_word;
            get(&all_words, 0, &min_word);
            int min_dist = edit_dist(s, min_word);
            for (int i = 1; i < list_size(&all_words); ++i) {
                char *cur;
                int cur_dist;
                get(&all_words, i, &cur);
                cur_dist = edit_dist(s, cur);
                if (cur_dist < min_dist) {
                    min_word = cur;
                    min_dist = cur_dist;
                }
            }
            if (min_dist < 4) {
                printf("You typed '%s'. Did you mean '%s'?\n", s, min_word);
            }
            c++;
        }
        free(s);
    }

    set_free(&dict);
    list_free(&all_words);
    fclose(fh);
    printf("Mistakes: %d\n", c);
    return 0;
}
```

Program 20 — Output

